

The Order of Hashing in Fiat-Shamir Schemes

Barbara Jiabao Benedikt ^(✉) and Marc Fischlin

Cryptoplexity, Technische Universität Darmstadt, Darmstadt, Germany
{barbara_jiabao.benedikt, marc.fischlin}@tu-darmstadt.de

Abstract. Fiat-Shamir signatures replace the challenge in interactive identification schemes by a hash value over the commitment, the message, and possibly the signer’s public key. This construction paradigm is well known and widely used in cryptography, for example, for Schnorr signatures and CRYSTALS-Dilithium. There is no “general recipe” for constructing Fiat-Shamir signatures regarding the inputs and their order for the hash computation, though, since the hash function is usually modeled as a monolithic random oracle. In practice, however, the hash function is based on the Merkle-Damgård or the sponge design.

Our work investigates whether there are advisable or imprudent input orders for hashing in Fiat-Shamir signatures. We examine Fiat-Shamir signatures with *plain* and *nested* hashing using Merkle-Damgård or sponge-based hash functions. We analyze these constructions in both classical and quantum settings. As part of our investigations, we introduce new security properties following the idea of quantum-annoyance of Eaton and Stebila (PQCrypto 2021), called *annoyance for user exposure and signature forgeries*. These properties ensure that an adversary against the hash function cannot gain a significant advantage when attempting to extend a successful attack on a single signature forgery to multiple users or to multiple forgeries of a single user. Instead, the adversary must create extra forgeries from scratch. Based on our analysis, we derive a simple rule: When using Fiat-Shamir signatures, one should hash the commitment *before* the message; all other inputs may be ordered arbitrarily.

Keywords: Fiat-Shamir Signatures · Merkle-Damgård Construction · Sponge Construction · Attacks · Quantum Algorithms · Schnorr Signatures · Dilithium

1 Introduction

In 1986, Fiat and Shamir [31] proposed a way to transform a three-move identification scheme into a signature scheme. The identification scheme consists of a protocol between a prover and a verifier in which the prover confirms her identity by showing that she knows the secret key sk to a public key pk . The structure of such canonical identification schemes consists of a commitment cmt sent by

©IACR ASIACRYPT 2025. This article is the full version of the article submitted by the authors to the IACR and to Springer-Verlag on 8th of September 2025.

the prover, a challenge ch returned by the verifier, and a response rsp computed by the prover, which the verifier eventually checks against the commitment, the challenge, and the public key.

The Fiat-Shamir transformation replaces the interactive generation of the challenge via a hash function H applied to the prover’s first commitment value cmt , the message m to be signed, and possibly further data. The original Fiat-Shamir transformation in [31] only included the message m and the commitment cmt (in this order) in the hashing step and considered the hash function H to be a monolithic random function.

1.1 What to Hash, and in Which Order?

There has been a debate about whether to include the public key into the hashing step of the Fiat-Shamir or not. Usually, the key constitutes the first part of the input to the hash function, which puts it in front of the message and potentially even in front of the commitment. This technique is thus often called “key-prefixing”. In other settings including public keys into the hash input is advisable, e.g., for domain separation in key encapsulation mechanisms [4] or to avoid malleability attacks on systems, such as the attack on the Helios voting system [6]. For signature schemes, including the the public key in the hash functions evaluation is, in general, suitable to prevent key-substitution attacks [49], ownership transfers [56], or further related properties [21]. Another reason to make the public key part of the input for signature schemes is to seemingly “alleviate concerns that several public keys could be attacked simultaneously” [8]. Key-prefixing was later proven by Bernstein [7] to affect so-called multi-user security positively for the case of Schnorr signatures, pointing out a flaw in a previous paper by Galbraith et al. [33], who claimed that multi-user security already tightly follows from single-user security, without key-prefixing. Later, Kiltz et al. [42] resurrected multi-user security of key-free Schnorr signature schemes.

If one does not assume that the hash function behaves like a monolithic random oracle, then the order of the message and commitment (and potentially the public key) may matter. In practice, we nowadays find two approaches to instantiate the hash function, either via the Merkle-Damgård construction [25,50] or via the sponge construction [9]. Unfortunately, neither of the two approaches is known to securely implement a monolithic random oracle (in the sense of indistinguishability [48,20]), even if assuming an idealized round function. For Merkle-Damgård it is known that, in general, the construction is not indistinguishable from a random oracle [20]. Yet, Dodis et al. [28] show that one can argue indistinguishability from so-called public-use random oracles, where the honest parties only apply the hash function to known inputs, as in the case of Fiat-Shamir signatures. Alas, we are not aware if this result extends to the quantum setting. In addition, their classical security bounds are quadratic in the number of queries and the length of inputs, such that we do not derive meaningful bounds if the adversary can make an exponential number of long queries. The situation is similar in the sponge case, where the security loss for indistinguishability is quadratic relative to the capacity in the classical case [9]. The best results for

the quantum case provide cubic security bounds, but usually restrict access to the round permutation [23], assume that it is a random function [22,24], or are only applicable to one-round sponges [60,16,47]. In summary, the general results about indistinguishability do not give a satisfying answer.

Regarding results about the order of hashing in Fiat-Shamir schemes, remember that Fiat and Shamir [31] originally placed m first and then cmt , but treated the hash function H as a random function. It has soon been discovered that milder requirements for the hash function may suffice if one puts the random commitment cmt first. Traces of this can already be found in Schnorr’s work [57,58], suggesting to hash cmt and m in this order. Later, Neven et al. [51] looked more closely into the specific hash function requirements for iterated hash function like SHA-2. They identified two properties of the hash function, called random-prefix preimage problem (RPP) and random-prefix second-preimage problem (RPSP), and show the (key-free) version of Schnorr’s scheme to be unforgeable in the generic group model under these assumptions. The proof of [51] was later shown to have a flaw [14,15].

The situation in standards is even more diverse. The order of hashing for the elliptic curve variant EdDSA [8] of Schnorr’s scheme is cmt , pk , and then m , which is also reflected in the IRTF standard [38]. ISO 14888-3 [37], on the other hand, defines Schnorr signatures with hash input cmt and m only. Other interesting cases are post-quantum secure signature schemes, like Picnic [18], Dilithium [29], MQDSS [19], and QTesla [2], which are all based on the Fiat-Shamir paradigm. The recently standardized Module-Lattice-Based Digital Signature Standard (ML-DSA) [54], originating from the Dilithium signature scheme [29], uses a “Fiat-Shamir with Aborts” technique [45]. It chains the hashes of the public key pk , the message m , and the commitment cmt in a nested way via the sponge-based hash function SHAKE-256 (or any other FIPS-approved hash function), i.e., it computes the challenge as $H(H(H(pk), m), cmt)$. This order contrasts sharply with the common approach to first input the random commitment. We also note that there is a version HashML-DSA in the standard [54] for signing pre-hashed messages $H(m)$ instead of m .

1.2 Our Contributions

Our starting point is the question of *what* to hash and *in which order* when applying the Fiat-Shamir transformation. We investigate this question in light of the security of the deployed hash function H . To make a statement about post-quantum signature schemes we are especially interested in questions regarding quantum attackers, which can evaluate the public hash function in superposition [11]. Another dimension we consider is the question if attacks against one user can be exploited to attack other users (similar to multi-user security), and if forgeries of a single message for one user allow to easily create forgeries of other messages (multi-forgery security).

A short answer to the question above about *what* to hash is easy: If possible, hash all data, cmt , pk , and m . This may be more complicated to fulfill in some

settings, though, especially on resource-bounded devices. First of all, if the public key is used in the signing process, then it must be present in the secret key.¹ Especially if public keys are large, then this can introduce further storage requirements. These can be mitigated by storing only parts or hashes of the public key, as for instance ML-DSA does with the public key’s trace $tr = H(pk)$. As a second example, smartcards sometimes support (partial) hashing of the message outside of the card, as for example specified in EN 419212-1 [17], to spare the transmission of the potentially long message to the card. In this case the signing process may only have access to $H(m)$ in the first place, as for example in HashML-DSA.

Classical and Quantum Attacks on the Hash Function. The more challenging question refers to the order of hash inputs. We systematically consider the different possibilities of combinations and, especially, how secure they are from a quantum attacker’s point of view. We assume that the underlying cryptographic problem, to compute sk from pk , is hard, and that the signing process of the Fiat-Shamir transformation is zero-knowledge and does not leak information about sk . We therefore look into potential attacks on the hash function in light of the input order for the signing data.

A starting point is the security requirements proposed by Neven et al. [51] for the hash function, RPP and RPSP. The RPP problem in the context of Fiat-Shamir signatures requires the adversary to first choose a hash value h , and after learning the commitment cmt to find a message m such that the hash evaluation yields h . The RPSP problem asks the adversary to determine a message m and, after receiving a commitment cmt , to find another message m' such that the hash evaluations collide. In the key-prefixing versions the adversary would receive pk at the outset. We are especially interested in quantum versions of the attacks against these properties.²

For the RPP notion, Kelsey and Kohno [40] already investigated classical attacks under the name herding or Nostradamus attacks. Benedikt et al. [5] lifted this to the quantum version. In contrast, for RPSP, the question of security is still open, and this problem immediately applies to our attack scenario against the order of hash inputs. In this context of investigating quantum RPSP attacks, we take a closer look at Trojan attacks [3,43] and extend it to the quantum and random-prefix setting. The original Trojan attack allows to generate collisions easily for a constrained set of potential prefixes, e.g., if we put a public key pk from a set of users first in the hashing step for Fiat-Shamir signatures. However, if the random commitment cmt comes first, then the set of possible prefixes is significantly larger and the original Trojan attacks [3,43] become infeasible. Furthermore, we prove that our new quantum attack is tight.

¹ Yet another alternative was to recompute the public key from the secret key, but this does not seem to be used in practice for efficiency reasons.

² The flaws in the security proofs based on these assumptions [14,15] are irrelevant for us since we only investigate attacks.

Quantum Annoyance. We also introduce another angle of consideration for the hash function, based on the work of Eaton and Stebila [30] about quantum annoyance. They defined this property for password-authenticated key exchange protocols, possibly based on classical security assumptions like the discrete logarithm assumption. For an adversary to successfully attack this scheme, she must execute an expensive quantum computation at least once for each user-password pair. In other words, the adversary does not have an advantage by running the quantum computer once and attacking multiple users-passwords simultaneously.

We adopt the idea of quantum annoyance for our signature schemes based on the Fiat-Shamir transformation. In the setting of a signature scheme, we observe that there are two hopefully quantum-annoying dimensions. On the one hand, the “user dimension” refers to the question if an expensive (quantum) pre-computation can be used to successfully attack several users simultaneously, i.e., to produce multiple forgeries for different users from the initial effort without requiring further quantum computations. The other “signature dimension” requires that the adversary cannot use a single quantum computation to produce forgeries for multiple messages for the same user. Of course, one can also consider the two dimensions concurrently in a two-dimensional space.

Recommendations. Taking into account possible attacks we establish the following simple rules for the order of the hash inputs:

1. To ensure quantum annoyance for user exposures and signature forgeries, the message m should not be hashed first.
2. A combination of the (quantum) Trojan attack and the quantum annoyance for signature forgeries reveals that the commitment cmt should be placed in front of the message m .

Altogether the rules leave four options—including the one that does not add the public key pk to the challenge.

M_1	M_2	M_3	Best Known Attack	Quantum Annoyance?
m	$*$	$*$	R-RSSP	not for user exposures
pk	m	cmt	RPSP	not for signature forgeries
pk	cmt	m	R-RPSP	✓
cmt	pk	m		
cmt	m	pk		
cmt	m	ε		

Fig. 1: Security of different input orders. Here, M_1, M_2 and M_3 denote the positions of the inputs in $H(M_1, M_2, M_3)$ with some recoverable encoding. Inputs $*$ refer to arbitrary inputs, and ε to an omitted input. The checkmarks only refer to the MD-based case. For sponge-based hash functions we were not able to prove quantum annoyance. However, our attacks still suggest that we should use the combinations which are highlighted in gray.

The Case of Dilithium and ML-DSA. As remarked above, Dilithium and ML-DSA deviate from our general recommendation on the order of hash inputs. We are not aware of any reason for this choice. Although this decreases the

security bounds, the choice of hash functions—more precisely, the sizes of the outputs and inner state—makes attacks infeasible. Dilithium and ML-DSA both use SHAKE-256 with capacity $c = 512$ as the hash function in the Fiat-Shamir transformation. Let us stress, however, that our best attacks significantly exceed practically relevant bounds in case of Dilithium.

For the pre-hash version of ML-DSA, only approved hash functions should be used for the pre-hashing step, e.g., according to FIPS 180 [32] or FIPS 202 [53]. Interestingly, the ML-DSA standard requires the hash function to have “at least λ bits of classical security strength against both collision and second preimage attacks”, ignoring quantum attacks. Indeed, the description of pre-hash ML-DSA explicitly mentions SHA-256 as an option [54, Alg. 4]. This would not only mean that quantum collision search is, at least theoretically, possible in $2^{85.3}$ steps, but also that such a successful collision search serves as an investment to simultaneously forge signatures for multiple users and, if using Joux’s multi-collision attack [39], to also simultaneously forge signatures for multiple messages.

2 Preliminaries

2.1 Signature Schemes

We use the common notation for signature schemes $S = (\text{Gen}, \text{Sign}, \text{Vrfy})$ where the probabilistic key generation algorithm $(\text{sk}, \text{pk}) \leftarrow \$ \text{Gen}(1^\lambda)$ samples a key pair, the probabilistic signature algorithms $\sigma \leftarrow \$ \text{Sign}_{\text{sk}}(\text{m})$ signs a message, and the (commonly deterministic) verification algorithm $\text{Vrfy}_{\text{pk}}(\text{m}, \sigma)$ outputs true or false. Correctness demands that for any key pair $(\text{sk}, \text{pk}) \leftarrow \$ \text{Gen}(1^\lambda)$, any message m and any signature $\sigma \leftarrow \$ \text{Sign}_{\text{sk}}(\text{m})$ we always have $\text{Vrfy}_{\text{pk}}(\text{m}, \sigma) = \text{true}$.

We say that key generation is *non-trivial* if the public keys produced by $\text{Gen}(1^\lambda)$ have super-logarithmic min-entropy. This is necessary in the following to prevent trivial forgery attacks, where an attacker executes the algorithm Gen multiple times, until she reproduces another user’s public key $\text{pk}' = \text{pk}$. Knowing the corresponding secret key, the adversary easily can produce signatures herself. All known practical schemes obey this property.

2.2 Fiat-Shamir Transformation

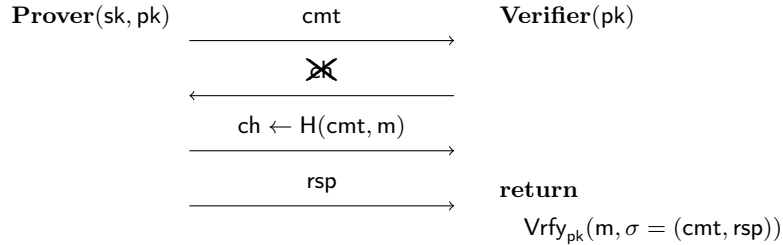


Fig. 2: The Fiat-Shamir transformation turns an identification into a signature scheme.

The Fiat-Shamir transformation allows to turn a (three-move) identification scheme into a signature scheme (cf. Fig. 2). In a three-move identification scheme, a prover (with a key pair (sk, pk)) is willing to prove her identity to the verifier by showing knowledge of sk . Therefore, she sends the first message—the commitment cmt —to the verifier. Using the received commitment and the prover’s public key pk , the verifier computes a challenge ch for the prover. The prover must provide a response rsp based on her secret key and the transcript of the conversation, which should finally convince the verifier to trust in the identification process. At the end of the interaction, the verifier outputs a decision bit whether she is convinced or not.

Turning this scheme into a signature scheme, where the prover is willing to prove that she has signed a message m . The challenge ch is in contrast determined by the prover. For the computation, she usually uses the commitment and the to-be-signed message, which she feeds into a hash function H . Afterwards, the prover responds to her self-chosen challenge and sends everything to the verifier. The final signature consists of the commitment cmt and the response rsp . If the commitment is *recoverable* from $\text{pk}, \text{ch}, \text{rsp}$, e.g., in the Schnorr signature scheme or in CRYSTALS-Dilithium, one can also put the challenge ch and the response rsp instead. We write $([\text{cmt}|\text{ch}], \text{rsp})$ for such a signature where either the commitment is given (and the challenge is computed by the verifier), or the challenge is given (and the verifier recovers the commitment to check). It is sometimes convenient to define a function $\text{C}(\text{pk}, \text{m}, \sigma) = \text{ch}$ outputting the challenge value, either by reading ch off the signature or by re-hashing the data. We say that the commitment is *non-trivial* if its min-entropy is super-logarithmic in the security parameter; unless stated otherwise we assume that this is the case (which is the usually case for practical schemes). In the end the verifier outputs a decision bit, whether she accepts the signature or not.

When inserting the data into the hash function, we assume some fixed encoding function $\text{encode}(i, x)$, taking an index $i \in \{1, 2, 3\}$ for the position and value x (either a string or an integer), and mapping these values to some string. An example is the encoding of string $\text{m} \in \{0, 1\}^{\leq 2^{64}}$ with a leading fixed-length string $\langle \text{m} \rangle_{64}$ denoting the message length, followed by the actual value as a string: $\text{encode}(i, \text{m}) = \langle \text{m} \rangle_{64} \parallel \text{m}$. For an empty input string $x = \varepsilon$ we require that $\text{encode}(i, \varepsilon) = \varepsilon$ for all i .

It is common to assume that the encoding function $\text{encode}(i, \cdot)$ is one-to-one for each i . We call this an *injective* encoding. The attacks in the following sections still work if the encoding function is not injective. For the positive results, however, we assume that this is indeed the case. It is sometimes also convenient to define the encoding to be *recoverable*, meaning that for any string $\text{encode}(1, x) \parallel \text{encode}(2, y) \parallel \text{encode}(3, z)$, we can unambiguously derive x, y , and z . Again, only some of the positive results need this extra property, but not our attacks. We usually denote $M_i = \text{encode}(i, x)$, where x is one of the potential inputs (commitment, message, public key, or empty string). We often omit the encoding and simply write $M_i = \text{cmt}$ with the understanding that $M_i = \text{encode}(i, \text{cmt})$.

We will mainly investigate *plain* and *nested* Fiat-Shamir signatures. In plain schemes a single evaluation of the hash function computes the challenge, encoding the potential inputs sequentially in an arbitrary order, and in nested schemes the challenge is computed by multiple and nested hash function evaluations:

Definition 1. We call a signature scheme $(\text{Gen}, \text{Sign}, \text{Vrfy})$ with non-trivial key generation a [plain|nested] Fiat-Shamir signature scheme if the scheme relies on the Fiat-Shamir transformation, i.e., the signature $\sigma = ([\text{cmt}|\text{ch}], \text{rsp})$ consists of the non-trivial commitment *cmt* (or, alternatively, of the challenge *ch* for a recoverable scheme) and the response *rsp*. For plain schemes, the challenge *ch* is of the form $\text{ch} \leftarrow \text{H}(M_1||M_2||M_3)$, and for nested schemes

$$\text{ch} \leftarrow \begin{cases} \text{H}(\text{H}(\text{H}(M_1)||M_2)||M_3), & \text{for } M_1, M_2, M_3 \neq \varepsilon, \\ \text{H}(\text{H}(M_1)||M_2), & \text{for } M_1, M_2 \neq \varepsilon, M_3 = \varepsilon, \end{cases}$$

where $M_1, M_2, M_3 \in \{m, \text{pk}, \text{cmt}, \varepsilon\}$ are pairwise distinct and ε refers to the empty string.³

For instance, Schnorr signatures are plain Fiat-Shamir signature schemes since the challenge is computed by $\text{ch} \leftarrow \text{H}(\text{cmt}||m)$ ($M_1 = \text{cmt}$, $M_2 = m$, and $M_3 = \varepsilon$), and CRYSTALS-Dilithium is nested since the challenge is $\text{ch} \leftarrow \text{H}(\text{H}(\text{H}(\text{pk})||m)||\text{cmt})$ ($M_1 = \text{pk}$, $M_2 = m$, and $M_3 = \text{cmt}$).

2.3 Hash Functions

In our work we focus on generic attacks on hash functions, which we then transfer to generic attacks on *plain and nested* Fiat-Shamir signature schemes. We do so, because it is sufficient to forge a *Fiat-Shamir signature* $\sigma = ([\text{cmt}|\text{ch}], \text{rsp})$ of a message *m*, if we succeed in forging *only* the challenge, i.e., we find another message *m'* for a given challenge $\text{ch} = \text{H}(\{m, \text{pk}, \text{cmt}\} \text{ in some order})$ such that $\text{ch}' = \text{H}(\{m', \text{pk}, \text{cmt}\} \text{ in some order}) = \text{ch}$. Then the signature σ is valid for both messages *m* and *m'*. This is because either we recover the same commitment *cmt* from *pk*, $\text{ch} = \text{ch}'$, and *rsp*, or compute the same challenge $\text{ch} = \text{ch}'$ from *pk*, *cmt*, *m* and *pk*, *cmt*, *m'*, and accept the response which depends on *sk*, *cmt*, $\text{ch} = \text{ch}'$ for both messages *m* and *m'*.

In general, the attacks that we present are working for iterative hash functions, i.e., the hash function takes the input message, divides it into message blocks of the same length, adds padding, and processes these blocks sequentially to compute the final hash value. We call the intermediate results of the block-wise calculations hash states. Nowadays such hash functions are usually built from the Merkle-Damgård (MD) construction [25,50] or sponge construction [9].

For sake of concreteness, we first assume that the hash function *H* is based on the MD construction $\text{H} : \{0, 1\}^* \rightarrow \{0, 1\}^n$. We treat sponge constructions later. The underlying compression function in the MD case is $h : \{0, 1\}^B \times \{0, 1\}^n \rightarrow$

³ The message *m* and the commitment *cmt* should of course be part of the input of the Fiat-Shamir transformation, but we do not formally require this here.

$\{0,1\}^n$, where B is the size of the message blocks and n is the size of the final hash value and the intermediate hash states. Moreover we consider a padding function $\text{pad} : \mathbb{N} \rightarrow \{0,1\}^{\leq B}$ which is used to transform any input message $\mathbf{m} \in \{0,1\}^*$ into a message $\mathbf{m} \parallel \text{pad}(|\mathbf{m}|) \in (\{0,1\}^B)^*$ aligned to block length B . Usually, the padding function takes the bit length of the message $|\mathbf{m}|$ as input and maps it to the string $10^d \parallel \langle |\mathbf{m}| \rangle$, where $d \in \mathbb{N}$ is chosen such that $\mathbf{m} \parallel 10^d \parallel \langle |\mathbf{m}| \rangle$ is aligned to block length B and is extended by at most one block. By $\langle i \rangle$ we denote a fixed-length binary encoding of i .

For simplicity we define the *iterated compression function* as

$$h^*(\mathbf{m}) := y_\ell, \text{ where } y_0 = \text{IV}, y_i = h(\mathbf{m}_i, y_{i-1}) \text{ for } i = 1, 2, \dots, \ell,$$

where $\mathbf{m} = \mathbf{m}_1 \parallel \mathbf{m}_2 \parallel \dots \parallel \mathbf{m}_\ell$, $\mathbf{m}_i \in \{0,1\}^B$, is the input divided into blocks of length B and IV is the (public) initialization vector. Then we can write the MD hash function as $H(\mathbf{m}) := h^*(\mathbf{m} \parallel \text{pad}(|\mathbf{m}|))$.

We make an assumption about “well-behaving” compression functions in MD constructions as Benedikt et al. [5]. Namely, we assume that the underlying compression function is β -balanced. Note that this is a reasonable property since random functions are β -balanced for $\beta = \frac{1}{2}$ with overwhelming probability for $B \gg n$ (cf. [5]).

Definition 2 (β -Balanced Compression Function). *Let $h : \{0,1\}^{B+n} \rightarrow \{0,1\}^n$, $B \geq n$, be the compression function of an MD-based hash function H . Then h is called β -balanced, where $\beta \in (0,1]$, if and only if for any $y \in \{0,1\}^n$ and any $z \in \{0,1\}^n$ the number of preimages satisfies $|h_y^{-1}(z)| \geq \beta \cdot 2^{B-n}$, where $h_y(\cdot) := h(\cdot, y)$.*

According to this definition it follows that for any β -balanced function h and for any $y \in \{0,1\}^n$ the function $h_y(\cdot)$ is surjective since $|h_y^{-1}(z)| \geq \beta \cdot 2^{B-n} > 0$ for any $z \in \{0,1\}^n$.

2.4 Hash Function Requirements

The first notion relevant to us was defined by Neven et al. [51], who considered the random prefix preimage (RPP) and -second preimage (RPSP) resistance. In our work we only investigate notions related to the second preimage resistance since these attacks resemble the real-world adversaries.

The so-called “Trojan” adversary determines a suffix S and gets a prefix chosen (uniformly) from a pre-known set of prefixes. In the context of signatures, this set of prefixes can, for example, contain the public keys $P_i = \text{pk}_i$ of (u -many) users. The adversary’s task is then to find another suffix S' such that the hash values of $P \parallel S$ and $P \parallel S'$ collide. The attack was analyzed in the classical setting [3,43], and refers to the following experiment:

Definition 3 (Trojan (RPSP) Attacker). *Let $\mathcal{A}_{RPSP}$ be a Trojan attacker. Then $\mathcal{A}_{RPSP}$ is successful against a hash function H if and only if for any pre-known set $\{P_i\}_{i=1}^u$, $u \in \mathbb{N}$, the following experiment $\text{Exp}_{H, \mathcal{A}_{RPSP}, \{P_i\}_{i=1}^u}^{RPSP}(\lambda)$ always returns 1:*

$$\begin{array}{l}
\textbf{Exp}_{\mathbf{H}, \mathcal{A}_{RPSP}, \{P_i\}_{i=1}^u}^{RPSP}(\lambda): \\
\hline
1: (st, S) \leftarrow \mathcal{A}_{RPSP}^{off}(1^\lambda, \{P_i\}_{i=1}^u) \quad // \text{ offline} \\
2: P \leftarrow \mathcal{S} \{P_i\}_{i=1}^u \\
3: S' \leftarrow \mathcal{A}_{RPSP}^{onl}(st, P, \{P_i\}_{i=1}^u) \quad // \text{ online} \\
4: \textbf{return} [\mathbf{H}(P\|S) = \mathbf{H}(P\|S') \wedge S \neq S']
\end{array}$$

Next, we consider the setting where the prefix contains the commitment **cmt**, so the set of prefixes is significantly larger. Since this is no longer feasible for a Trojan attacker, we consider an adversary who knows only a maximum length ℓ_P of the prefix in advance, leading us to the so-called Random-Trojan adversary:

Definition 4 (Random-Trojan (R-RPSP) Attacker). *Let $\mathcal{A}_{R-RPSP}$ be a Random-Trojan attacker. Then $\mathcal{A}_{R-RPSP}$ is successful against a hash function $\mathbf{H}$ if and only if for any $\ell_P \in \mathbb{N}$ the following experiment $\textbf{Exp}_{\mathbf{H}, \mathcal{A}_{R-RPSP}, \ell_P}^{R-RPSP}(\lambda)$ always returns 1:*

$$\begin{array}{l}
\textbf{Exp}_{\mathbf{H}, \mathcal{A}_{R-RPSP}, \ell_P}^{R-RPSP}(\lambda): \\
\hline
1: (st, S) \leftarrow \mathcal{A}_{R-RPSP}^{off}(1^\lambda) \quad // \text{ offline} \\
2: P \leftarrow \mathcal{S} \{0, 1\}^{\leq \ell_P} \\
3: S' \leftarrow \mathcal{A}_{R-RPSP}^{onl}(st, P) \quad // \text{ online} \\
4: \textbf{return} [\mathbf{H}(P\|S) = \mathbf{H}(P\|S') \wedge S \neq S']
\end{array}$$

Moreover we consider another notion which follows the same structure, but the randomness appears as the suffix—the random suffix second preimage (R-RSSP) resistance. We call the R-RSSP attacker the Hypnotist attacker since we can imagine her as someone who can tell two different stories ($P \neq P'$) with the same level of plausibility and credibility if continued with suffix S .

Definition 5 (Hypnotist (R-RSSP) Attacker). *Let $\mathcal{A}_{R-RSSP}$ be a (Random-)Hypnotist attacker. Then $\mathcal{A}_{R-RSSP}$ is successful against a hash function $\mathbf{H}$ if and only if for any $\ell_S \in \mathbb{N}$ the following experiment $\textbf{Exp}_{\mathbf{H}, \mathcal{A}_{R-RSSP}, \ell_S}^{R-RSSP}(\lambda)$ always returns 1:*

$$\begin{array}{l}
\textbf{Exp}_{\mathbf{H}, \mathcal{A}_{R-RSSP}, \ell_S}^{R-RSSP}(\lambda): \\
\hline
1: (st, P) \leftarrow \mathcal{A}_{R-RSSP}^{off}(1^\lambda) \quad // \text{ offline} \\
2: S \leftarrow \mathcal{S} \{0, 1\}^{\leq \ell_S} \\
3: P' \leftarrow \mathcal{A}_{R-RSSP}^{onl}(st, S) \quad // \text{ online} \\
4: \textbf{return} [\mathbf{H}(P\|S) = \mathbf{H}(P'\|S) \wedge P \neq P']
\end{array}$$

3 General Attack Structure

As already mentioned, we are looking into different possibilities to order the message **m**, the public key **pk** and the commitment **cmt** for hashing, and evaluate their security, both in the classical and in the quantum setting.

3.1 General Attack Model

We consider a general attack model where the adversary knows the public key(s) and has access to the signing oracles of all users. Our adversary is restricted to classical oracle-queries since we assume that the users—even in the post-quantum scenario—have a classical computer and respond only to classical queries. The run time $T[\mathcal{A}]$ of an adversary $\mathcal{A}$ is given by the number of round function evaluations.

In our model we divide the ℓ -forgery attacker $\mathcal{A}_\ell$ into an offline and an online phase. In the offline phase the adversary only has access to the public keys of users and prepares a useful message m . At the beginning of the online phase, after the adversary managed to foist the dedicated message on the users, she learns signatures for m under all user keys, and then tries to find ℓ fresh forgeries with the help of the signatures. We note that one could more generally allow to learn further signatures afterwards, granting the adversary also access to signing oracles of all users, but most the attacks already work with the single-message attack such that we keep the structure here simple.

We also assume that the adversary in the experiment succeeds with probability negligibly close to 1. Most of the attacks we discuss satisfy this. It simplifies the comparison of run times between different algorithms, especially when defining annoyance and comparing 1-forgery attacks with ℓ -forgery attacks. In this case, executing the 1-forgery attacker ℓ times may otherwise significantly reduce the success probability if the success probability of the 1-forgery is small.

Finally, we only consider here attacks against the hash function in the sense that the adversary needs to find matching hash values. That is, we assume that the adversary re-uses a given challenge in a signature for its attack but for a new message. This is specific to our setting here, as we consider the underlying problem computing sk from pk to be infeasible; this could be omitted for a general notion. In the definition below we thus use the previously defined function $C(pk, m, \sigma) = ch$ returning the challenge value for a signature.

Definition 6 (General Attack Model). *The adversary $\mathcal{A}_\ell$ is asked to forge $\ell \in \mathbb{N}$, $\ell = \text{poly}(\lambda)$, signatures of users $\{U_i\}_{i=1}^u$ with given key pairs $(sk_i, pk_i) \leftarrow \text{Gen}(1^\lambda)$. In the first step, the offline phase, the challenger hands the public keys $\{pk_i\}_{i=1}^u$ to the adversary $\mathcal{A}_\ell$. The adversary outputs a message m and state information. In the online phase, the adversary receives signatures for the message and, eventually, $\mathcal{A}_\ell$ outputs its forgery attempts $\{(m_i, \sigma_i, p_i)\}_{i=1}^\ell$, where p_i identifies the public key pk_{p_i} . Adversary $\mathcal{A}_\ell$ is successful if she outputs ℓ message-signature-key pointer tuples $(m_i, \sigma_i, p_i)_{i=1}^\ell$ such that*

- the signature σ_i is valid for any $i \in \{1, \dots, \ell\}$, i.e., $\text{Vrfy}_{pk_{p_i}}(m_i, \sigma_i) = \text{true}$,
- the forgeries targeting a single user correspond to distinct messages, i.e., the tuples $(m_i, p_i) \neq (m_j, p_j)$, $i \neq j \in \{1, \dots, \ell\}$, are pairwise different,
- the message $m_i \neq m$ is “fresh” for any $i \in \{1, \dots, \ell\}$.

The general experiment (with additional oracle access to the signature oracles) is as follows:

ℓ -Forgery Attack $Exp_{S, \mathcal{A}_\ell, u}^{\ell\text{-forgery}}(\lambda)$:

```

1: for  $i = 1, 2, \dots, u$  do  $(\text{sk}_i, \text{pk}_i) \leftarrow \text{Gen}(1^\lambda)$ 
2:  $(st, m) \leftarrow \mathcal{A}_\ell^{\text{off}}(1^\lambda, \{\text{pk}_i\}_{i=1}^u)$ 
3: for  $i = 1, 2, \dots, u$  do  $\sigma_{m,i} \leftarrow \text{Sign}_{\text{sk}_i}(m)$ 
4:  $\{(m_i, \sigma_i, p_i)\}_{i=1}^\ell \leftarrow \mathcal{A}_\ell^{\text{onl}}(st, \{\sigma_{m,i}\}_{i=1}^u, [\{\text{Sign}_{\text{sk}_i}(\cdot)\}_{i=1}^u])$  // optional oracle access
5: // let  $\mathcal{Q}_i$  be the set of queried messages to  $\text{Sign}_{\text{sk}_i}$ , including  $m$ 
6: // let  $\mathcal{C}_i$  be the set of challenges of created signatures, including the one for  $m$ 
7: return 1 if
8:    $\text{Vrfy}_{\text{pk}_{p_i}}(m_i, \sigma_i) = \text{true}$  for all  $i = 1, 2, \dots, \ell$ 
9:    $(m_i, p_i) \neq (m_j, p_j)$  for all  $i \neq j$ 
10:   $m_i \notin \mathcal{Q}_{p_i}$  and  $\mathcal{C}(\text{pk}_{p_i}, m_i, \sigma_i) \in \mathcal{C}_{p_i}$  for all  $i = 1, 2, \dots, \ell$ 

```

We assume that the adversary's success probability is negligibly close to 1. We call an ℓ -forgery attack

- an ℓ -multi-forgery attack if a single user is affected, i.e., all key pointers $p_1 = \dots = p_\ell$ refer to the same user,
- an ℓ -multi-user attack if ℓ users are affected, i.e., all key pointers in $\{p_i\}_{i=1}^\ell$ are pairwise distinct.

The ℓ -multi-forgery attack refers to the extreme dimension in which the adversary tries to forge signatures for many messages m_j for the same user after having obtained a signature of this user for an appropriately chosen message m . This means that the (assumed expensive) precomputation time allows to create multiple forgeries for the targeted user easily.

The ℓ -multi-user attack is the other extreme dimension in which the adversary forges signatures for another message for each user after having obtained a signature under the chosen message m of this user. In other words, the precomputation time pays off to expose multiple users. Any other attack is in between the two extreme dimensions.

One could also introduce an “off-offline” phase of the adversary $\mathcal{A}_\ell$ in which she does not even have access to the public keys. This would correspond to settings where the hash function, e.g., SHAKE, has been designed earlier, and the actual signature scheme significantly later. Then the adversary could have used the delay in time till the first public keys of the signature schemes appear to collect information about the hash function. But this seems to be only relevant in the scenario where the message is hashed first.

3.2 Classical and Quantum Annoyance

The phenomenon that the effort of an adversary can be different if she uses the quantum computer for each user once or once for all users is also described by Eaton and Stebila [30], who called it quantum-annoying. Considering

the ℓ -forgery adversary in our setting, it is reasonable to shift the definition of quantum-annoyance into the multi-user and the multi-forgery setting of signature schemes, as well as *classical* annoyance in both settings. Hereby, we are only interested in non-negligible differences in run time.

We capture annoyance below by demanding that there exists an 1-forgery adversary $\mathcal{A}_1$, denoting the best attack strategy to create a single forgery, such that any ℓ -forgery adversary $\mathcal{A}_\ell$ cannot do better than creating ℓ single forgeries. In other words, adversary $\mathcal{A}_\ell$ cannot profit from shifting computations between forgeries. Depending on the setting whether we consider a multi-user or a multi-forgery adversary, we also use ℓ_u and ℓ_σ as notation instead of ℓ .

Definition 7 (Classical- and Quantum-Annoyance). *Let $u \in \mathbb{N}$, $u = \text{poly}(\lambda)$. A signature scheme is called **[classical/quantum]-annoying for user exposures**, iff for any $\ell_u \leq u$ the effort of running a 1-forgery attack and an ℓ_u -multi-user attack are relatively the same, i.e., there exists a [classical/quantum] 1-forgery adversary $\mathcal{A}_1$ such that for any [classical/quantum] ℓ_u -multi-user adversary $\mathcal{A}_{\ell_u}$:*

$$T[\mathcal{A}_{\ell_u}] = \Omega(\ell_u \cdot T[\mathcal{A}_1]),$$

where $T[\cdot]$ is the run time of the algorithms.

*A signature scheme is **[classical/quantum]-annoying for signature forgeries**, iff for any $\ell_\sigma = \text{poly}(\lambda)$ the effort of running a 1-forgery attack and an ℓ_σ -multi-forgery attack are relatively the same, i.e., there exists a [classical/quantum] 1-forgery adversary $\mathcal{A}_1$ such that $T[\mathcal{A}_{\ell_\sigma}] = \Omega(\ell_\sigma \cdot T[\mathcal{A}_1])$ for any [classical/quantum] ℓ_σ -multi-forgery adversary $\mathcal{A}_{\ell_\sigma}$.*

We further refine the definition of *annoyance* with respect to the online phase and an offline phase that can be used for pre-computations. The pre-computations may be carried out before the adversary is able to make the users sign a target message, after which additional computations may be necessary in the online phase once the signature details become known. Our refined notion of strong-[classical/quantum]-annoyance hence weighs in the online phase more significantly. We call a scheme strongly annoying if even an adversary who has made pre-computations for a long period cannot gain a relative advantage from forging multiple signatures instead of a single one.

Definition 8 (Strong Classical- and Quantum-Annoyance). *Let $u \in \mathbb{N}$, $u = \text{poly}(\lambda)$. A signature scheme is **strongly-[classical/quantum]-annoying for user exposures** resp. **signature forgeries**, iff the scheme is [classical/quantum]-annoying for user exposures resp. signature forgeries and fulfills: For all $\ell_u \leq u$ resp. $\ell_\sigma = \text{poly}(\lambda)$, there exists a [classical/quantum] 1-forgery adversary $\mathcal{A}_1$, such that $T^{\text{onl}}[\mathcal{A}_\ell] = \Omega((\ell_u \text{ resp. } \ell_\sigma) \cdot T^{\text{onl}}[\mathcal{A}_1])$ for any [classical/quantum] ℓ_u -multi-user adversary $\mathcal{A}_\ell$ resp. ℓ_σ -multi-forgery adversary $\mathcal{A}_\ell$, where $T^{\text{onl}}[\cdot]$ is the run time of the online phase.*

Sometimes, proving annoyance is challenging. The reason is that time-space trade-offs act in favor of the adversary: Pre-computing large tables may speed up the individual search and amortize the overall complexity. Therefore we occasionally relax Definition 7 and 8 into a version that allows for some slack:

Definition 9 (ε -[Classical|Quantum]-Annoyance). Let $u \in \mathbb{N}$, $u = \text{poly}(\lambda)$. A signature system is called **[strongly]- ε -[classical|quantum]-annoying for user exposures** resp. **signature forgeries**, $\varepsilon \leq 1$, iff for any $\ell_u \leq u$ resp. $\ell_\sigma = \text{poly}(\lambda)$ the effort of running a 1-forgery attack and an ℓ_u -multi-user resp. ℓ_σ -multi-forgery attack is relatively the same w.r.t. a parameter ε , i.e., there exists a [classical|quantum] 1-forgery adversary $\mathcal{A}_1$ such that for any [classical|quantum] ℓ_u -multi-user resp. ℓ_σ -multi-forgery adversary $\mathcal{A}_\ell$:

$$\begin{aligned} T[\mathcal{A}_\ell] &= \Omega((\ell_u \text{ resp. } \ell_\sigma)^\varepsilon \cdot T[\mathcal{A}_1]), \\ [\wedge T^{\text{onl}}[\mathcal{A}_\ell] &= \Omega((\ell_u \text{ resp. } \ell_\sigma)^\varepsilon \cdot T^{\text{onl}}[\mathcal{A}_1])). \end{aligned}$$

We usually assume that ε is at least some constant, ideally close to 1.

4 Lower Bounds

Before diving into successful Trojan and Hypnotist attackers, we first recall lower bounds for the run time of the best attacks, assuming an idealized compression function which is modeled as a random oracle. Since a successful Trojan and Hypnotist adversary also outputs a collision of the compression function, a lower bound for finding collisions [59] (with constant probability) is also a lower bound of Trojans and Hypnotists:

$$\Omega([\text{Trojan}|\text{Hypnotist}] \text{ Attack}) \geq \Omega(\text{Find Collision}) = \begin{cases} \Omega(2^{n/2}) & \text{classically,} \\ \Omega(2^{n/3}) & \text{quantumly.} \end{cases}$$

This means that if we can present attacks which match these lower bounds, then they are optimal. Ideally, however, one requires less strict requirements than collision resistance of the hash function, allowing also to give improved security bounds. Put differently, if we can present a collision-finding attack with run time close to the above lower bounds for a specific order of inputs, then one should view this order as sub optimal.

4.1 Lower Bounds in the Classical Setting

For the notions of annoyance we are also interested in the complexity of finding ℓ different collision pairs in random functions. Dinur [26] has shown that, in the classical setting, to find ℓ collisions in a random function $f : [N] \rightarrow [N]$ one needs roughly $T \geq \tilde{\Omega}(\ell \cdot (N/S)^{1/2})$ steps, where S is the size of the available space. The bound is optimal (up to poly-logarithmic factors) as it matches the parallel collision search algorithm of van Oorschot and Wiener [55]. In our setting, the state information passed from the offline to the online stage gives a lower bound for the space requirement. One could in principle thus use this lower bound for a refined view on time-space lower bounds for our general attack structure. But since we currently do not impose any restrictions on the size of this space we can use a more direct bound for arbitrary functions:

Theorem 10. *Any classical algorithm for finding ℓ different collision pairs in a random function $f : [M] \rightarrow [N]$, $M > N$, with success probability at least $2/3$ must satisfy $T \geq \ell^{1/2} N^{1/2}$.*

Proof. After T steps and queries to the random function, there are at most $T^2/2$ many possible collision pairs. Each such input pair has a probability of $1/N$ to collide, such that the expected number E of collisions is bounded by $E \leq T^2/2N$. Hence, the probability to have $\ell \geq T^2/N$ collisions (even not necessarily pairwise distinct ones) is, by Markov's inequality, given by

$$\Pr[\ell \geq T^2/N] \leq \Pr[\ell \geq 2E] \leq \frac{1}{2}.$$

It follows that for an adversary to succeed with probability at least $2/3$ in finding ℓ distinct collision pairs in T steps, one must have $\ell \leq T^2/N$. This yields the claimed bound. $\square$

4.2 Lower Bounds in the Quantum Setting

Recently, Hamoudi and Magniez [36] gave a bound for time-space trade-offs for finding multiple collisions in the quantum setting. Their bound basically says that such a search algorithm requires $T^3 S \geq \Omega(\ell^3 \cdot 2^n)$. Hamoudi and Magniez [36] also present an algorithm achieving run time $T = \tilde{O}(\ell^{2/3} N^{1/3})$ for space up to $S = \tilde{O}(\ell^{2/3} N^{1/3})$. A stronger lower bound, without reference to space, was given by Liu and Zhandry:

Theorem 11 ([44, Thm. 5]). *Any quantum algorithm for finding ℓ different collision pairs in a random function $f : [M] \rightarrow [N]$, $M > N$, with constant success probability must make at least $\Omega(\ell^{2/3} N^{1/3})$ queries.*

5 Where to Place the Message?

We first argue that putting the message as the first hash function input is not optimal. It allows for collision-finding attacks with overwhelming parts carried out in the offline phase, and also supports attacks against multiple users.

5.1 Not at the Beginning

Let us consider combinations starting with the message $M_1 = \mathbf{m}$ and continuing the hash evaluation with $M_2, M_3 \in \{\mathbf{pk}, \mathbf{cmt}, \varepsilon\}$. The adversary's attack strategy is to find an inner hash collision for the message part such that any signature for one message, for which the signing oracle samples a random commitment $\mathbf{cmt}$ for the signature, immediately constitutes a valid signature for the other message as well. Accordingly, we are in the setting of a Hypnotist attack, where the message

part M_1 corresponds to a prefix for which the adversary later receives a suffix. We can thus use the following hypnotist adversary:

$$\frac{\mathcal{A}_{\text{R-RSSP}}^{\text{off}}(1^\lambda):}{1 : \text{ \textbf{return} } (P', P) \leftarrow \text{collision of } h_{\text{IV}}(\cdot)} \quad \frac{\mathcal{A}_{\text{R-RSSP}}^{\text{onl}}((P', P), S):}{1 : \text{ \textbf{return} } P'}$$

Concerning the run time, in the classical case we perform a birthday attack on $h_{\text{IV}}(\cdot)$ to find a collision in the offline phase which requires $\mathcal{O}(2^{n/2})$ evaluations of the compression function. In the quantum case, we use the BHT-algorithm (cf. Proposition 12), which requires $\mathcal{O}(2^{n/3})$ evaluations of h , to find a collision of $h_{\text{IV}}(\cdot)$ in the offline phase. In both settings, the online phase requires constant time $\mathcal{O}(1)$. Hence, the algorithms are also optimal.

We recall here some facts about the quantum search since we need this later to evaluate security of the order of hashing for Fiat-Shamir schemes.

Proposition 12 ((Adapted) BHT-Algorithm [13]). *Let $y \in \{0, 1\}^n$ be a hash state, and $h : \{0, 1\}^B \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ be a β -balanced compression function, $\beta \in (0, 1]$. Then there exists a quantum algorithm making $\mathcal{O}(\beta^{-1/3} \cdot 2^{n/3})$ quantum queries to h which eventually outputs a collision of $h_y(\cdot)$, i.e.,*

$$\text{BHT}(h_y(\cdot)) \rightarrow (m, m') \in \{0, 1\}^B \times \{0, 1\}^B$$

such that $h_y(m) = h(m, y) = h(m', y) = h_y(m')$.

The BHT-algorithm makes use of a generalization of Grover's algorithm [35] as stated as follows.

Theorem 13 (Grover, [12]). *Let $F : X \rightarrow Y$ be a function, $N := |X|$ be the cardinality of the domain, $y_0 \in Y$ be fixed and*

$$t := |F^{-1}(y_0)| = |\{x \in X \mid F(x) = y_0\}|$$

be the cardinality of the preimage of y_0 under F . If $t \geq 1$, then $\text{Grover}(F, y_0)$ eventually outputs an $x \in X$ with $F(x) = y_0$ after $\mathcal{O}(\sqrt{N/t})$ evaluations of F .

Proof of Prop. 12. Given $y_0 \in Y := \{0, 1\}^n$, the BHT-algorithm proceeds as follows. Let $k \in \mathbb{N}$. Sample k message blocks $\mathbf{m}_1, \dots, \mathbf{m}_k$, and compute the hash states $\{h_{y_0}(\mathbf{m}_i)\}_{i=1}^k$. If two messages $\mathbf{m}_i$ and $\mathbf{m}_j$ collide, output $(\mathbf{m}_i, \mathbf{m}_j)$. If not, apply $\text{Grover}(F, 1)$ (cf. Thm. 13) where $F : \{0, 1\}^B \rightarrow \{0, 1\}$ is defined as follows

$$F(\mathbf{m}) = 1 \quad :\Longleftrightarrow \quad \exists i \in \{1, \dots, k\} \text{ such that } h_{y_0}(\mathbf{m}) = h_{y_0}(\mathbf{m}_i) \text{ and } \mathbf{m} \neq \mathbf{m}_i,$$

i.e., F outputs 1 for messages, which collide with one of the sampled messages $\{\mathbf{m}_i\}_{i=1}^k$. Finally, the algorithm searches for $i \in \{1, \dots, k\}$ such that $\mathbf{m}_i$ collides with $\mathbf{m}$, and outputs $(\mathbf{m}, \mathbf{m}_i)$.

The presampling requires k evaluations of h , and Grover another $\mathcal{O}(\sqrt{N/t})$ evaluations, where $N = 2^B$ and $t = |F^{-1}(1)| \geq (\beta \cdot 2^{B-n} - 1) \cdot k$ because of the balancedness of h . Accordingly, we get an overall run time of $\mathcal{O}(k + (\beta \cdot k)^{-1/2} \cdot 2^{n/2})$, and $\mathcal{O}(\beta^{-1/3} \cdot 2^{n/3})$ if $k = \beta^{-1/3} \cdot 2^{n/3}$. $\square$

If β is constant, this algorithm matches the lower bound $\Omega(2^{n/3})$.

5.2 Mounting an ℓ -Multi-User Attack

Let us first consider how a 1-forgery attacker $\mathcal{A}_1$ makes use of $\mathcal{A}_{\text{R-RSSP}}$ from above. She starts with a query to $\mathcal{A}_{\text{R-RSSP}}^{\text{off}}$ and receives the output (P', P) . Next $\mathcal{A}_1$ uses P as message $\mathbf{m}$ and sends it to a signing oracle of her choice, e.g., $\text{Sign}_{\text{sk}_1}(\cdot)$ to get a signature $\sigma = ([\text{cmt}|\text{ch}], \text{rsp})$ of P . Afterwards $\mathcal{A}_1$ wins the game by outputting $(P', \sigma, 1)$ which is a valid Fiat-Shamir signature in both modes, since $h^*(P) = h^*(P')$ (needed for `mode = plain`) and $H(P) = H(P')$ (for `mode = nested`) hold.

For $1 < \ell \leq u$, adversary $\mathcal{A}_\ell$ is doing the same but for ℓ different users simultaneously:

$$\begin{array}{c|c} \mathcal{A}_\ell^{\text{off}}(1^\lambda, \{\text{pk}_i\}_{i=1}^u): & \mathcal{A}_\ell^{\text{onl}}(st, \{\sigma_{P,i}\}_{i=1}^u): \\ \hline 1 : & (P', P) \leftarrow \mathcal{A}_{\text{R-RSSP}}^{\text{off}}(1^\lambda) \quad 1 : \text{ return } \{(P', \sigma_{P,i}, i)\}_{i=1}^\ell \\ 2 : & \text{ return } (st, \mathbf{m}) = ((P', P), P) \end{array}$$

Note that $\mathcal{A}_\ell$ is an adversary who manages to compromise ℓ users by running $\mathcal{A}_{\text{R-RSSP}}^{\text{off}}$ once and receiving ℓ different signatures. In particular, the running time $\mathcal{O}(2^{n/2} \text{ resp. } n/3 + \ell)$ does not change significantly with increasing ℓ :

$$T[\mathcal{A}_\ell] = \Theta\left(\left\{\frac{2^{n/2}}{2^{n/3}}\right\} + \ell\right) \notin \Omega\left(\ell \cdot \left\{\frac{2^{n/2}}{2^{n/3}}\right\} + \ell\right).$$

Thus, we get the following corollary:

Corollary 14. *Any [plain/nested] Fiat-Shamir signature scheme starting with $M_1 = m$ is not [classical/quantum]-annoying for user exposures.*

5.3 Mounting an ℓ -Multi-Forgery Attack

In case $\ell = 1$, our 1-multi-forgery attacker acts like the 1-multi-user adversary from above, so let us consider the case $\ell > 1$. First, $\mathcal{A}_\ell$ proceeds as $\mathcal{A}_1$ to obtain the first forgery $(P', \sigma_1, 1)$. In contrast to before, she cannot send P multiple times to $\text{Sign}_{\text{sk}_1}(\cdot)$ because this does not lead to fresh forgeries with messages $\mathbf{m} \neq P'$, which are needed according to Def. 6. For winning the ℓ -multi-forgery game, $\mathcal{A}_\ell$ could rerun $\mathcal{A}_{\text{R-RSSP}}^{\text{off}}$ ℓ -times to obtain ℓ distinct collisions $\{(P_i, P'_i)\}_{i=1}^\ell$ of h , and output $\{(P'_i, \sigma_{P'_i, 1}, 1)\}_{i=1}^\ell$. However, this would be the trivial solution to run the 1-forgery multiple times.

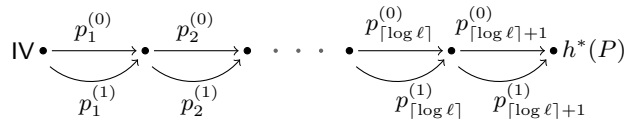


Fig. 3: Joux's multi-collision [39] to obtain an $(\ell + 1)$ -collision $(P, P_1, \dots, P_\ell)$ of h^* , where $P = p_1^{(0)} \parallel \dots \parallel p_{\lceil \log \ell \rceil + 1}^{(0)}$ and $P_i = p_1^{(i_1)} \parallel \dots \parallel p_{\lceil \log \ell \rceil + 1}^{(i_{\lceil \log \ell \rceil + 1})}$ ($i_1 i_2 \dots i_{\lceil \log \ell \rceil + 1}$ is the binary encoding of i).

$\mathcal{A}_\ell^{\text{off}}(1^\lambda, \{\text{pk}_i\}_{i=1}^u):$
1 : Build a $\lceil \log_2 \ell \rceil + 1$ multi-collision // as in Fig. 3
2 : **return** $(st, \mathbf{m}) = ((P, P_1, \dots, P_\ell), P)$

$\mathcal{A}_\ell^{\text{onl}}(st, \{\sigma_i\}_{i=1}^u):$
1 : **for** $i \in \{1, \dots, \ell\}$ **do**
2 : $i_1 i_2 \dots i_{\lceil \log_2 \ell \rceil + 1} \leftarrow$ binary encoding of i
3 : $P_i \leftarrow p_1^{(i_1)} \parallel \dots \parallel p_{\lceil \log_2 \ell \rceil + 1}^{(i_{\lceil \log_2 \ell \rceil + 1})}$
4 : **return** $\{(P_i, \sigma_i, 1)\}_{i=1}^\ell$

Fig. 4: Multi-Forgery adversary $\mathcal{A}_\ell$ using Joux’s multi-collisions.

It is more efficient to obtain an $(\ell + 1)$ -collision $(P, P_1, \dots, P_\ell)$ of h^* using Joux’s multi-collision attack [39] (cf. Fig. 3) and to act as follows:

For building these multi-collisions $\mathcal{A}_\ell$ applies either the birthday attack or the BHT-algorithm multiple times, requiring $\mathcal{O}(\log_2 \ell \cdot 2^{n/2}$ resp. $n/3$) evaluations of h . Having the multi-collisions, $\mathcal{A}_\ell$ can extract at least $\ell + 1$ distinct prefixes $(P, P_1, \dots, P_\ell)$ by taking different paths in Fig. 3. Adversary $\mathcal{A}_\ell$ then asks for a signature for P under key sk_1 , receives σ_1 , and wins the multi-forgery game by outputting $\{(P_i, \sigma_i, 1)\}_{i=1}^\ell$. Thus,

$$T[\mathcal{A}_\ell] = \mathcal{O}\left(\log_2 \ell \cdot \left\{\frac{2^{n/2}}{2^{n/3}}\right\}\right) \notin \Omega(\ell \cdot T[\mathcal{A}_1]) = \Omega\left(\ell \cdot \left\{\frac{2^{n/2}}{2^{n/3}}\right\}\right), \quad (1)$$

and we get the following corollary:

Corollary 15. *Any [plain/nested] Fiat-Shamir signature scheme starting with $M_1 = \mathbf{m}$ is not [classical/quantum]-annoying for signature forgeries.*

However, there is a difference between the [classical/quantum]-annoyance for user exposures and signature forgeries. With increasing ℓ , the effort for ℓ -multi-user attacks does not change significantly, whereas for the ℓ -multi-forgery attacks, there exists an upper bound of the effort which scales with ℓ (cf. Inequality (1)). With our relaxed version of annoyance, allowing $T[\mathcal{A}_\ell] = \Omega(\ell^\varepsilon \cdot T[\mathcal{A}_1])$, we obtain:

Proposition 16. *Any [plain/nested] Fiat-Shamir signature scheme starting with $M_1 = \mathbf{m}$ is at most $\log_\ell \log_2 \ell$ -[classical/quantum]-annoying for signature forgeries, i.e., the scheme is ε -[classical/quantum]-annoying where $0 \leq \varepsilon \leq \log_\ell \log_2 \ell$, but not strongly annoying.*

Proof. Consider a [plain/nested] Fiat-Shamir signature scheme starting with $M_1 = \mathbf{m}$. Then any ℓ -multi-forgery adversary $\mathcal{A}_\ell$ has a lower bound, which is at most runtime of the adversary described above:

$$T[\mathcal{A}_\ell] = \Omega(\ell^\varepsilon \cdot T[\mathcal{A}_1]) \leq \Omega\left(\log_2 \ell \cdot \left\{\frac{2^{n/2}}{2^{n/3}}\right\}\right).$$

Accordingly there exists a value $0 \leq \varepsilon \leq \log_\ell \log_2 \ell$ —since $\log_2 \ell = \ell^{\log_\ell \log_2 \ell}$ —proving that the signature scheme is ε -[classical|quantum]-annoying.

However, the scheme is not strongly-[classical|quantum]-annoying since $T^{\text{onl}}[\mathcal{A}_1] = T^{\text{onl}}[\mathcal{A}_\ell] = \mathcal{O}(1)$. $\square$

According to Corollary 14 and Corollary 15, starting with the message is generally not recommended, so we derive our first rule, stating that the message should not be hashed first. This matches the intuition that finding collisions in the message-part otherwise suffices to break the scheme’s security.

1st RULE: Do not start with the message $\mathbf{m}$ for hashing.

6 How to Order Message and Commitment?

In this section, we will answer how to relatively order the message $\mathbf{m}$ and the commitment $\mathbf{cmt}$. The result will be the following rule:

2nd RULE: Use the commitment $\mathbf{cmt}$ before the message $\mathbf{m}$ for hashing.

Nevertheless, let us start with the case of having message $\mathbf{m}$ before the commitment $\mathbf{cmt}$ and with $\text{mode} = \text{plain}$. Concerning the so-far derived rule (of not starting with $\mathbf{m}$), the only possible combination starts with the public key $M_1 = \mathbf{pk}$ followed by message $M_2 = \mathbf{m}$ and commitment $M_3 = \mathbf{cmt}$.

In that case, our adversary gets a random prefix ($\mathbf{pk}$) and a random suffix ($\mathbf{cmt}$) after committing to a message $\mathbf{m}$. However, a successful Trojan attacker—the one who gets a random prefix—is already sufficient to forge a signature for this scheme if she outputs suffixes of the same length ($|S| = |S'|$). This is because the two suffixes $\mathbf{m}$ and $\mathbf{m}'$ she outputs fulfill the equations $h^*(\mathbf{pk}||\mathbf{m}) = h^*(\mathbf{pk}||\mathbf{m}')$ and $h^*(\mathbf{pk}||\mathbf{m}||\mathbf{cmt}) = h^*(\mathbf{pk}||\mathbf{m}'||\mathbf{cmt})$, where $\mathbf{pk}$ is the given prefix and $\mathbf{cmt}$ an arbitrary commitment. Accordingly the adversary—taking her messages $\mathbf{m}$ and $\mathbf{m}'$ —also runs a successful attack against the full challenge

$$\text{ch} = H(\mathbf{pk}||\mathbf{m}||\mathbf{cmt}) = H(\mathbf{pk}||\mathbf{m}'||\mathbf{cmt}) = \text{ch}'$$

for any $\mathbf{cmt}$. This is true since the padding remains the same if $|\mathbf{m}| = |\mathbf{m}'|$. For $\text{mode} = \text{nested}$, we can proceed in the same way with $H(\mathbf{pk})$ as prefix since

$$\text{ch} = H(H(H(\mathbf{pk})||\mathbf{m})||\mathbf{cmt}) = H(H(H(\mathbf{pk})||\mathbf{m}')||\mathbf{cmt}) = \text{ch}'.$$

The work of Andreeva et al. [3] and Kortelainen et al. [43] propose a Trojan attack in a setting where the prefix is sampled from an a-priorily known set. Fortunately this fits the scenario of this section—the random prefix is one of the public keys given by the challenger beforehand.

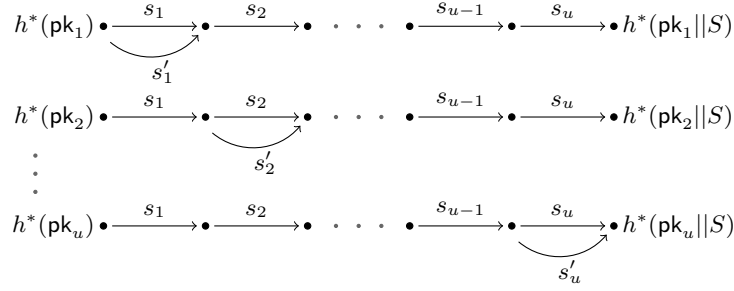


Fig. 5: The Trojan attack [3].

6.1 Trojan Attack

Let $\{\mathbf{pk}_i\}_{i=1}^u$ be the set of possible prefixes with $|\mathbf{pk}_i| = |\mathbf{pk}_j|$ for any $i, j \in \{1, \dots, u\}$. Assume for the beginning that each $\mathbf{pk}_i$ is aligned to the block length (and $\text{mode} = \text{plain}$). Adversary $\mathcal{A}_{\text{RPSP}}$ first searches for a collision (s_1, s'_1) of h^* starting with $\mathbf{pk}_1$, i.e., $h^*(\mathbf{pk}_1||s_1) = h^*(\mathbf{pk}_1||s'_1)$ (cf. Fig. 5). Next, she does the same but with $\mathbf{pk}_2||s_1$ as prefix, then for $\mathbf{pk}_3||s_1||s_2$ and so on. Afterwards she commits to $S = s_1||s_2||\dots||s_u$ so that, later on when she is demanded to compromise user i , she outputs $S' = s_1||\dots||s'_i||\dots||s_u$ as suffix. Both suffixes S and S' fulfill $h^*(\mathbf{pk}_i||S) = h^*(\mathbf{pk}_i||S')$.

Assuming the public keys $\{\mathbf{pk}_i\}_{i=1}^u$ are not aligned to block length, there exists integers $k, r \in \mathbb{N}$ such that $|\mathbf{pk}_i| = k \cdot B + r$ and $0 < r < B$. Then $\mathcal{A}_{\text{RPSP}}$ samples $s_0 \leftarrow \{0, 1\}^{B-r}$ such that $\mathbf{pk}_i||s_0$ is aligned to block length and proceeds as previously, but with $\mathbf{pk}_i||s_0$ as prefix instead of $\mathbf{pk}_i$ and $S = s_0||s_1||\dots||s_u$ instead of $S = s_1||\dots||s_u$. For $\text{mode} = \text{nested}$, she uses $H(\mathbf{pk}_i)$ as prefix.

$\mathcal{A}_{\text{RPSP}}^{\text{off}}(1^\lambda, \{P_i\}_{i=1}^u):$	$\mathcal{A}_{\text{RPSP}}^{\text{onl}}((st, S), P, \{P_i\}_{i=1}^u):$
1 : $S, st \leftarrow \varepsilon$	1 : Parse st as $s'_1 \dots s'_u$,
2 : for $i = \{1, \dots, u\}$ do	and S as $s_1 s_2 \dots s_u$
3 : $(s_i, s'_i) \leftarrow \text{collision of } h_{h^*(P_i S)}(\cdot)$	2 : Find $i \in \{1, \dots, u\}$ s.t. $P = P_i$
4 : $st \leftarrow st s'_i$ and $S \leftarrow S s_i$	3 : return
5 : return $[(st, S)]$	$S' \leftarrow s_1 \dots s_{i-1} s'_i s_{i+1} \dots s_u$

For the run time we obtain both in the classical and quantum setting, the adversary needs to find u collisions. In the classical case this requires $\mathcal{O}(2^{n/2})$ steps via the birthday attack; in the quantum case this requires $\mathcal{O}(2^{n/3})$ steps via BHT-algorithm. Observe further that $\mathcal{A}_{\text{RPSP}}^{\text{off}}$ requires $\sum_{i=1}^u (\ell_{\mathbf{pk}} + i - 1)$ evaluations for line 3, where $\ell_{\mathbf{pk}}$ is the prefix block size, and every other step runs in constant time $\mathcal{O}(1)$. Putting the pieces together, $\mathcal{A}_{\text{RPSP}}^{\text{off}}$ has an overall run time of

$$\mathcal{O}\left(\left(\sum_{i=1}^u \ell_{\mathbf{pk}} + i - 1\right) + u \cdot \left\{ \frac{2^{n/2}}{2^{n/3}} \right\}\right) = \mathcal{O}\left(u \cdot \ell_{\mathbf{pk}} + u^2 + u \cdot \left\{ \frac{2^{n/2}}{2^{n/3}} \right\}\right),$$

and $\mathcal{A}_{\text{RPSP}}^{\text{onl}}$ a run time of $\mathcal{O}(1)$ to search for the right prefix. Under the assumption that $\ell_{\mathbf{pk}}$ is constant and $u = \text{poly}(\lambda)$, we get an overall run time of $\mathcal{O}(u \cdot 2^{n/2} \text{ resp. } n/3)$.

6.2 Mounting an ℓ -Multi-User Attack

$\mathcal{A}_\ell^{\text{off}}(1^\lambda, \{\text{pk}_i\}_{i=1}^u)$:

```

1 : if (mode = plain) then  $(st, S) \leftarrow \mathcal{A}_{\text{RPSP}}^{\text{off}}(1^\lambda, \{\text{pk}_i\}_{i=1}^\ell)$ 
2 : if (mode = nested) then  $(st, S) \leftarrow \mathcal{A}_{\text{RPSP}}^{\text{off}}(1^\lambda, \{H(\text{pk}_i)\}_{i=1}^\ell)$ 
3 : return  $(st, S)$ 

```

$\mathcal{A}_\ell^{\text{onl}}(st, \{\sigma_i\}_{i=1}^u)$:

```

1 : for  $i \in \{1, \dots, \ell\}$  do
2 :   if (mode = plain) then  $S_i \leftarrow \mathcal{A}_{\text{RPSP}}^{\text{onl}}((st, S), \text{pk}_i, \{\text{pk}_i\}_{i=1}^u)$ 
3 :   if (mode = nested) then  $S_i \leftarrow \mathcal{A}_{\text{RPSP}}^{\text{onl}}((st, S), H(\text{pk}_i), \{H(\text{pk}_i)\}_{i=1}^u)$ 
4 : return  $\{(S_i, \sigma_i, i)\}_{i=1}^\ell$ 

```

Fig. 6: An ℓ -multi-user adversary for schemes of the form $(M_1, M_2, M_3) = (\text{pk}, \text{m}, \text{cmt})$.

Let $\ell \in \mathbb{N}$ with $\ell \leq u = \text{poly}(\lambda)$. Consider the ℓ -multi-user adversary from Figure 6, with slight differences according to the hashing **mode** of the signature scheme. Note that $\mathcal{A}_{\text{RPSP}}^{\text{off}}$ “only” gets ℓ of the u public keys to restrict the computational effort. The resulting adversary $\mathcal{A}_\ell$ is successful since all signatures $\{\sigma_i\}_{i=1}^\ell$ are valid and any message $\{S_i\}_{i=1}^u$ is “fresh” ($\neq S$). The run time consists of one execution of $\mathcal{A}_{\text{RPSP}}^{\text{off}}$ and ℓ executions of $\mathcal{A}_{\text{RPSP}}^{\text{onl}}$, resulting in an overall run time of $\Theta(\ell \cdot 2^{n/2} \text{ resp. } n/3)$.

Using our relaxed notion of annoyance (Definition 9), we can show that $\mathcal{A}_\ell$ is almost optimal (up to a constant factor in the exponent). The adversary $\mathcal{A}_\ell$ also shows that the scheme is not strongly annoying, where we compare the online run times:

Theorem 17. *Any [plain/nested] Fiat-Shamir signature scheme of the form $(M_1, M_2, M_3) = (\text{pk}, \text{m}, \text{cmt})$ is $[\frac{1}{2}\text{-classical}/\frac{2}{3}\text{-quantum}]$ -annoying for user exposures, but not strongly annoying.*

Proof. Let $\ell \in \mathbb{N}$ with $\ell \leq u = \text{poly}(\lambda)$. Note, that an ℓ -multi-user attack has the same lower bound as finding ℓ -many 2-collisions. The reason is that the intermediate values of $\{h^*(\text{pk}_i)\}_{i=1}^u$ (resp. $\{h^*(H(\text{pk}_i))\}_{i=1}^u$ if **mode** = **nested**) are pairwise different with overwhelming probability, since the public keys are honestly generated with sufficiently high min-entropy and a collision (among keys or within the first hash function iterations) is thus unlikely. Hence, any successful attack must satisfy $T^2 \geq \Omega(\ell 2^n)$ in the classical case, and $T \geq \Omega(\ell^{2/3} 2^{n/3})$ in the quantum case (cf. Theorems 10 and 11). The claim now follows for $\varepsilon = 1/2$ resp. $2/3$, because the best 1-forgery attacker has run time $\Theta(2^{n/2} \text{ resp. } n/3)$.

However, the scheme is not strongly-[classical|quantum]-annoying for user exposures since both the adversaries $\mathcal{A}_\ell$ (from Figure 6), as well its 1-forgery version $\mathcal{A}_1$, have online run time $T^{\text{onl}}[\mathcal{A}_1] = T^{\text{onl}}[\mathcal{A}_\ell] = \mathcal{O}(1)$. $\square$

6.3 Mounting an ℓ -Multi-Forgery Attack

Let $\ell \in \mathbb{N}$ with $\ell = \text{poly}(\lambda)$, and $\mathcal{A}_\ell$ an ℓ -multi-forgery adversary. In that case, $\mathcal{A}_\ell$ fixes an user with her public key and searches—in contrast to before—for

an $(\ell + 1)$ -collision of a “single function”. Accordingly $\mathcal{A}_\ell$ takes advantage of Joux’s multi-collision attack [39] (cf. Fig. 7), and acts in the same way as the adversary in Fig. 4. Therefore, Inequality (1) holds here as well, and leads us to the following result:

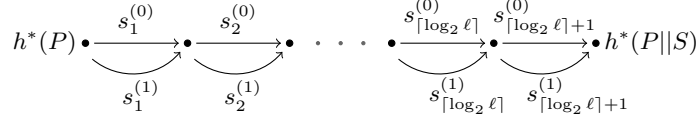


Fig. 7: An ℓ -multi-forgery attack using Joux’s [39] multi-collisions, where $P = \text{pk}_1$ if $\text{mode} = \text{plain}$ and $P = H(\text{pk}_1)$ if $\text{mode} = \text{nested}$. All resulting signatures are valid, and all suffixes $\{S, S_1, \dots, S_\ell\}$ —which correspond to different paths in Fig. 7—are pairwise different.

Corollary 18. *Any [plain/nested] Fiat-Shamir signature scheme of the form $(M_1, M_2, M_3) = (\text{pk}, m, \text{cmt})$ is at most $\log_\ell \log_2 \ell$ -[classical/quantum]-annoying for signature forgeries, but not strongly annoying.*

7 Recommended Ordering in Fiat-Shamir Schemes

According to our derived rules, the setting where the prefix is random (and not presampled from an a-priorily known set), is the most demanding one for adversaries. Therefore the commitment cmt must be part of the prefix, what leaves us with four possibilities to build our signature scheme:

- (1) $M_1 = \text{pk}$, $M_2 = \text{cmt}$, $M_3 = m$, (3) $M_1 = \text{cmt}$, $M_2 = m$, $M_3 = \text{pk}$, and
- (2) $M_1 = \text{cmt}$, $M_2 = \text{pk}$, $M_3 = m$, (4) $M_1 = \text{cmt}$, $M_2 = m$.

Note that these combinations correspond to the “best-achievable” setting where MD-constructions can be used. In each of the cases, the prefix determines the *last input hash state* before the MD-construction processes the message m —this state can be seen as (randomized) IV (or salt) of the hash function which only takes the message. For this specific hash function, the adversary cannot prepare a useful collision *without* knowing the randomized IV . Since the IV is unknown in the offline phase, she can’t choose the message based on precomputed collisions. As a result, the adversary faces a second preimage problem rather than a collision-finding problem, which must be solved online after the state IV is known. Therefore the lower bound in Thm. 10 for collision-finding is not applicable in this context: not every collision solves a second preimage problem.

7.1 ...in the Classical Setting

Since we are unaware of classical tight bounds of finding (multiple) second preimage(s), we are not able to prove classical-annoyance for the combinations (1)-(4). However, we have good evidence to believe in ε -annoyance with $\varepsilon \approx 1$. As we said, using the MD-construction with randomized IV is the most demanding challenge

for an adversary, and, since MD-constructions are used in many cryptographic protocols (e.g., in TLS 1.3), this is a setting where we trust in the security.

We know a lower bound of finding a single second preimage, but it is unclear how this extends to the case of solving multiple preimage tasks. This should be especially considered under the setting where an offline adversary may perform pre-computations. There is some indication that this problem remains hard in our setting of recommended hash orders. For this note that every successful second preimage attack is also a successful collision attack on H . In that case—if the message length is bounded by a constant, $|m| \in \mathcal{O}(1)$ —we can consult the following result, extending a previous result by Dodis et al. [27]:

Theorem 19 ([34, Thm. 1.1]). *Let $B \in \mathcal{O}(1)$ be constant, h be a random oracle and $(\mathcal{A}_1, \mathcal{A}_2)$ a two-staged adversary parametrized by (space) S and (time) T . $\mathcal{A}_1$ computes with unbounded access to h an S -bit advice for $\mathcal{A}_2$. $\mathcal{A}_2$ gets the advice and a random IV , makes at most T queries to h , and attempts to find a B -block collision (m, m') for IV . Both $\mathcal{A}_1$ and $\mathcal{A}_2$ are unbounded in the running time. The advantage of any $(\mathcal{A}_1, \mathcal{A}_2)$ is bounded by $\tilde{O}(ST/2^n + T^2/2^n)$.*

As mentioned above, one can view the hash order with a commitment cmt in the beginning as determining the initialization vector IV of the hash function after the offline phase. In particular, $(\mathcal{A}_1, \mathcal{A}_2)$ is successful, if either $ST \approx 2^n$ or $T \approx 2^{n/2}$. Assuming that $\mathcal{A}_1$ has at least time complexity $\Omega(S)$ to output a helpful S -bit advice, any successful adversary $(\mathcal{A}_1, \mathcal{A}_2)$ requires $\tilde{O}(2^{n/2})$ queries.

In case of unbounded message sizes—if m has block size $B = 2^{n/2}$ —one can run the second preimage attack of Kelsey and Schneier [41] using expandable messages, to get a run time $\tilde{O}(2^{n/2})$. However, this case can easily be avoided by suggesting a maximum length of messages, which can be signed within a signature.

7.2 ...in the Quantum setting

As in the classical setting, we have good reasons to trust in MD-constructions with randomized IV . However, in the quantum setting we are able to prove tight bounds for finding (multiple) second preimage(s) for MD-constructions if we allow unbounded collisions. This bound matches the tight bound for finding (multiple) collisions.

Therefore we investigate Random-Trojan adversaries (cf. Def. 4) who take advantage of long messages. Although, once more, one can easily avoid these attacks in practice, we nevertheless present our new Random-Trojan attack in the following, since it achieves optimal run time and is useful for multi-forgery attacks.

A Simple Attack. The Random-Trojan adversary first commits on a suffix S and receives then the challenge—the prefix P —to find another $S' \neq S$ with $H(P||S) = H(P||S')$. In our case, the prefix consists of the commitment and possibly the public key. For example, for a scheme with $(M_1, M_2, M_3) = (\text{cmt}, m, \varepsilon)$ we have $P = \text{cmt}$ if $\text{mode} = \text{plain}$ or $P = H(\text{cmt})$ if $\text{mode} = \text{nested}$. Let us

assume that P is aligned to block length B , i.e., there is an integer ℓ_P such that $|P| = \ell_P \cdot B$. We discuss later how our attack works for a prefix $P \in \{0, 1\}^*$ of arbitrary length.

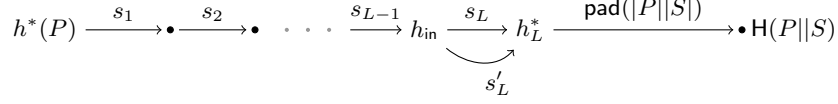


Fig. 8: Our simple attack to find another suffix $S' = s_1||s_2||\dots||s'_L$ given a random prefix $P \in \{0, 1\}^{\ell_P \cdot B}$ which is aligned to block length.

In the offline phase, $\mathcal{A}_{\text{R-RPSP, simple}}^{\text{off}}$ outputs an arbitrary suffix S of block length $L \in \mathbb{N}$, i.e., $S = s_1||\dots||s_L$ with $\{s_i\}_{i=1}^L \subset \{0, 1\}^B$. Accordingly, the composite message $P||S \in \{0, 1\}^{(\ell_P+L) \cdot B}$ is also aligned to block length. Next, $\mathcal{A}_{\text{R-RPSP, simple}}^{\text{onl}}$ searches for an alternative suffix S' for P via Grover (cf. Thm. 13) as shown in Fig. 8. Hereby she aims for a suffix S' of the same length ($|S'| = |S|$), that only differs in the last message block. This has the advantage that the padding does not change if we substitute one message block with another one.

$\mathcal{A}_{\text{R-RPSP, simple}}^{\text{off}}(1^\lambda, L)$:	$\mathcal{A}_{\text{R-RPSP, simple}}^{\text{onl}}((\varepsilon, S), P)$:
1 : Choose a suffix $S \in \{0, 1\}^{L \cdot B}$	1 : Parse S as $s_1 s_2 \dots s_L$
2 : return $[(\varepsilon, S)]$	2 : $h_{in} \leftarrow h^*(P s_1 \dots s_{L-1})$
	3 : $s'_L \leftarrow \text{Grover}(F_{h_{in}, s_L}, 1) \quad // F \text{ as in 2}$
	4 : return $S' := s_1 \dots s_{L-1} s'_L$

Consider the help function $F_{h_{in}, m'} : \{0, 1\}^B \rightarrow \{0, 1\}$, where $h_{in} \in \{0, 1\}^n$ is a hash state and $m' \in \{0, 1\}^B$ a message, which is defined by

$$F_{h_{in}, m'}(m) = 1 \quad :\Longleftrightarrow \quad h(m, h_{in}) = h(m', h_{in}) \text{ and } m \neq m', \quad (2)$$

i.e., the function $F_{h_{in}, m'}$ outputs 1 for a message m , if $m \neq m'$ and if m leads us from the input hash state h_{in} to the same output hash state $h(h_{in}, m')$ as m' .

Accordingly, $\text{Grover}(F_{h_{in}, m'}, 1)$ with $h_{in} := h^*(P||s_1||\dots||s_{L-1})$ and $m' := s_L$ eventually outputs a message s'_L , which ensures that $S' = s_1||\dots||s_{L-1}||s'_L$ is indeed an alternative since

$$H(P||S) = h^*(P||S||\text{pad}(|P||S|)) = \underbrace{h(\text{pad}(|P||S|))}_{=\text{pad}(|P||S'|)} \underbrace{h(s_L, h_{in})}_{=h(s'_L, h_{in})} = H(P||S').$$

Run time. $\mathcal{A}_{\text{R-RPSP, simple}}^{\text{off}}$ runs in time $\mathcal{O}(1)$, whereas $\mathcal{A}_{\text{R-RPSP, simple}}^{\text{onl}}$ requires $\ell_P + (L - 1)$ evaluations of h in line 2 and $\mathcal{O}(\beta^{-1/2} \cdot 2^{n/2})$ evaluations in line 3.

Let us consider the general case $P \in \{0, 1\}^*$. Then there exists $k, r \in \mathbb{N}_0$ s.t. $r < B$ and $|P| = k \cdot B + r$ and we can parse P as $p_1||p_2||\dots||p_k||p_{k+1}$, where $\{p_i\}_{i=1}^k \subset \{0, 1\}^B$ and $p_{k+1} \in \{0, 1\}^r$. Consider $\mathcal{A}_{\text{R-RPSP, simple}}^{\text{off}}$ who chooses a suffix $S \in \{0, 1\}^{(L+1) \cdot B}$, which $\mathcal{A}_{\text{R-RPSP, simple}}^{\text{onl}}$ parses as $s_0||s_1||\dots||s_{L+1}$ where $s_0 \in \{0, 1\}^{B-r}$, $\{s_i\}_{i=1}^L \subset \{0, 1\}^B$ and $s_{L+1} \in \{0, 1\}^r$ —possibly the empty string if $r = 0$. Afterward $\mathcal{A}_{\text{R-RPSP, simple}}^{\text{onl}}$ proceeds as stated in line 2-4, leading to a successful Random-Trojan attack (cf. Fig. 9). This attack also requires $\mathcal{O}(L + \ell_P + \beta^{-1/2} \cdot 2^{n/2})$ evaluations of h .

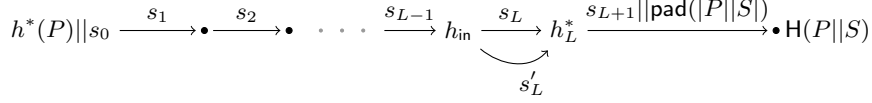


Fig. 9: Our simple attack to find another suffix $S' = s_0 \parallel s_1 \parallel \dots \parallel s'_L \parallel s_{L+1}$ given a random prefix $P \in \{0, 1\}^*$ of arbitrary length. Note that since $P||S$ is possibly not aligned to block length B , the message $s_{L+1} \parallel \text{pad}(|P||S|)$ (last arrow) is either of block length one or two.

An Enhanced and Optimal Attack. The idea to improve the simple attack is to allow that the alternative message block can replace any of the blocks $\{s_i\}_{i=1}^L$ (cf. Fig. 10). This raises the number of possible suffixes S' and speeds up **Grover**.

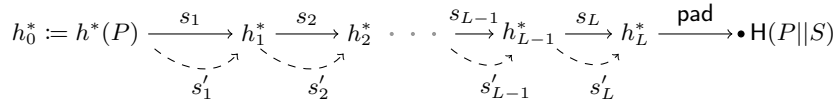


Fig. 10: Our enhanced attack to find another suffix S' given a random prefix P .

Assume, $P \in \{0, 1\}^{\ell_P \cdot B}$ is aligned to block length. If not, we can proceed as before. We define $F_{h_0, \dots, h_L, m_1, \dots, m_L} : \{0, 1\}^B \rightarrow \{0, 1\}$ as follows

$$F_{h_0, \dots, m_L}(m) = 1 \iff \exists i \in \{1, \dots, L\} : h_i = h(m, h_{i-1}^*) \text{ and } m \neq m_i. \quad (3)$$

That is, for L -many input hash states and message blocks the function outputs 1 for a message, which connects one of the hash states $h_i, i \in \{0, \dots, L-1\}$, with the subsequent hash state h_{i+1} . In our case, we apply **Grover** to F with input hash states $\{h_i^*\}_{i=0}^L$, and message blocks $\{s_i\}_{i=1}^L$ to find an $s'_i, i \in \{0, \dots, L-1\}$, as in Fig. 10. Since $s'_i \neq s_i$ holds by definition of F , the enhanced adversary $\mathcal{A}_{\text{R-RPSP}}$ succeeds and eventually finds another suffix S' .

$\mathcal{A}_{\text{R-RPSP}}^{\text{off}}(1^\lambda, L):$	$\mathcal{A}_{\text{R-RPSP}}^{\text{onl}}((\epsilon, S), P):$
1 : Choose a suffix $S \in \{0, 1\}^{L \cdot B}$	1 : Parse S as $s_1 \parallel s_2 \parallel \dots \parallel s_L$
2 : return $[(\epsilon, S)]$	2 : $h_0^* \leftarrow h^*(P)$
	3 : for $i = 1, \dots, L$ do $h_i^* \leftarrow h(s_i, h_{i-1}^*)$
	4 : $s' \leftarrow \text{Grover}(F_{h_0^*, \dots, h_L^*, s_1, \dots, s_L}, 1)$ // F as in 3
	5 : Find $i \in \{1, \dots, L\}$ s.t. $h_i^* = h(s', h_{i-1}^*)$
	6 : return $S' := s_1 \parallel \dots \parallel s_{i-1} \parallel s' \parallel s_{i+1} \parallel \dots \parallel s_L$

Run time. $\mathcal{A}_{\text{R-RPSP}}^{\text{off}}$ has run time $\mathcal{O}(1)$. The search space is $\{0, 1\}^B$, and the expected number of possible alternative message blocks is $L \cdot \beta \cdot 2^{B-n}$. Thus **Grover** (line 4) runs in time $\mathcal{O}(L^{-1/2} \cdot \beta^{-1/2} \cdot 2^{n/2})$. Furthermore $\ell_P + L$ additional evaluations are required to compute the hash states $h_0^*, \dots, h_L^*$ (line 2-3), and possibly another $\mathcal{O}(L)$ evaluations in line 5.

In total, the enhanced attack requires $\mathcal{O}(\ell_P + L + L^{-1/2} \cdot \beta^{-1/2} \cdot 2^{n/2})$ evaluations of h , so that the attack is optimized for $L = 2^{n/3}$. If ℓ_P, β are constant, the attack then runs in time $\mathcal{O}(2^{n/3})$ and is optimal. However this attack can

easily be avoided in practice by suggesting a maximum length significantly below $L = 2^{n/3}$ of signed messages.

7.3 Mounting an ℓ -Multi-User Attack

Let $\ell \leq u$ and $\mathcal{A}_\ell$ a multi-user adversary. In the offline phase, $\mathcal{A}_\ell$ runs $(\varepsilon, S) \leftarrow \mathcal{A}_{\text{R-RPSP}}^{\text{off}}$ and sends S to the signing oracle of ℓ different users. In the online phase, she uses the received signatures to recover the commitment cmt and computes the prefix (as previously described) based on cmt and—possibly—the corresponding public key. For each of the ℓ prefixes, she runs $\mathcal{A}_{\text{R-RPSP}}^{\text{onl}}$ together with $((\varepsilon, S), P)$. Given the adversary $\mathcal{A}_\ell$, we can easily prove:

Theorem 20. *Any [plain/nested] Fiat-Shamir signature scheme, where cmt is hashed before m , is $\frac{2}{3}$ -quantum-annoying for user exposures.*

Proof. Let $\ell \in \mathbb{N}$ with $\ell \leq u = \text{poly}(\lambda)$. Note that a commitment cmt carries sufficiently large min-entropy such that the commitments of different users do not collide, except with negligible probability. For any of the four possible orders, including the one where only the commitment but not the public key enters, this leads us to the same setting as in the proof of Thm. 17 where we needed ℓ -many 2-collisions starting from different hash states. In this case the starting points of the functions are $\{h^*(\text{cmt}_i)\}_{i=1}^\ell$ (resp. $\{H(\text{cmt}_i)\}_{i=1}^\ell$ if $\text{mode} = \text{nested}$), or $\{h^*(\text{pk}_i || \text{cmt}_i)\}_{i=1}^\ell$ or $\{h^*(\text{cmt}_i || \text{pk}_i)\}_{i=1}^\ell$. This results in a quantum lower bound of $\Omega(\ell^{2/3} \cdot 2^{n/3})$ which matches with the run time of our previously introduced Random-Trojan attack (when taking into account the exponent $\varepsilon = 2/3$). $\square$

7.4 Mounting an ℓ -Multi-Forgery Attack

Let $\ell = \text{poly}(\lambda)$ and $\mathcal{A}_\ell$ a multi-forgery adversary. First she fixes an user with her public key, chooses a message m and queries only a single signature σ for m . This is already sufficient since the corresponding commitment cmt (of σ) can be reused over multiple forgeries as long as we stick to the same user.

The situation has changed: the multi-user setting requires ℓ -many 2-collisions, whereas the multi-forgery setting only a single $(\ell + 1)$ -collision.

Theorem 21. *Any [plain/nested] Fiat-Shamir signature scheme, where cmt is hashed before m , is $\frac{2}{3}$ -quantum-annoying for signature forgeries.*

Proof. Let $\ell \in \mathbb{N}$ with $\ell = \text{poly}(\lambda)$, and $\mathcal{A}_\ell$ an ℓ -multi-forgery adversary.

First we prove that multi-forgery attacks have the lower bound $\Omega(\ell^{2/3} \cdot 2^{n/3})$. Note, that the final forgeries $\{\sigma_i = ([\text{cmt}_i | \text{ch}_i], \text{rsp}_i)\}_{i=1}^\ell$ can either correspond to different or the same commitments. The one extreme, where all commitments are pairwise different, is equivalent to the scenario in Thm. 20 and requires to find ℓ -different 2-collisions. This can be done in time $\Omega(\ell^{2/3} \cdot 2^{n/3})$.

On the other hand, we have the other extreme, where all commitments $\text{cmt}_1 = \dots = \text{cmt}_\ell$ are identical. In this case, there is a common starting point

P and $\mathcal{A}_\ell$ searches for an $(\ell + 1)$ -collision under the function h^* of the form $(P||S, P||S_1, \dots, P||S_\ell)$, where S is the committed suffix from the offline phase.

However, an $(\ell + 1)$ -collision of the iterated compression function h^* implies ℓ -many 2-collisions of the underlying compression function h : if we draw the messages $P||S$ and $P||S_1$ as paths, these paths first diverge and then collide at some point—giving us our first 2-collision. For each $i = 2, \dots, \ell$, the message $P||S_i$ is another path which eventually collides with one of the previous-drawn paths $P||S, \dots, P||S_{i-1}$, thus we get in total (at least) ℓ -many 2-collisions (cf. Thm. 11). Therefore $\mathcal{A}_{\text{R-RPSP}, \ell}$ matches the lower bound of finding ℓ -many 2-collisions.

From now on, it is sufficient to come up with an adversary who matches the lower bound to prove the claim. Consider the following Random-Trojan attacker:

$\mathcal{A}_{\text{R-RPSP}, \ell}^{\text{off}}(1^\lambda, L)$:	$\mathcal{A}_{\text{R-RPSP}, \ell}^{\text{onl}}((\epsilon, S), P)$:
1 : Choose a suffix $S \in \{0, 1\}^{L \cdot B}$	1 : Parse S as $s_1 s_2 \dots s_L$
2 : return $[(\epsilon, S)]$	2 : $h_0^* \leftarrow h^*(P)$
	3 : for $i = 1, \dots, L$ do $h_i^* \leftarrow h(s_i, h_{i-1}^*)$
	4 : for $i = 1, \dots, \ell$ do
	5 : repeat $s'_i \leftarrow \text{Grover}(F_{h_0^*, \dots, h_L^*, s_1, \dots, s_L}, 1)$ // F as in 3
	6 : until $s'_i \notin \{s'_1, \dots, s'_{i-1}\}$
	7 : Find $j_i \in \{1, \dots, L\}$ s.t. $h_{j_i}^* = h(s'_i, h_{j_i-1}^*)$
	8 : $S_i \leftarrow s_1 \dots s_{j_i-1} s'_i s_{j_i+1} \dots s_L$
	9 : return $\{S_i\}_{i=1}^\ell$

This adapted Random-Trojan adversary $\mathcal{A}_{\text{R-RPSP}, \ell}$ requires

$$\mathcal{O}(\ell_P + L + \ell \cdot L^{-1/2} \cdot \beta^{-1/2} \cdot 2^{n/2})$$

evaluations of h , so that the attack gets optimal for $L = \ell^{2/3} \cdot 2^{n/3}$. Then the run time—if ℓ_P, β are constant—is $\mathcal{O}(\ell^{2/3} \cdot 2^{n/3})$. Thus,

$$\ell \cdot T[\mathcal{A}_{\text{R-RPSP}, 1}] = \Theta(\ell \cdot 2^{n/3}) \text{ and } T[\mathcal{A}_{\text{R-RPSP}, \ell}] = \Omega(\ell^{2/3} \cdot 2^{n/3}). \quad \square$$

At first glance, there might seem to be no notable difference between the combinations $(M_1, M_2, M_3) = (\text{pk}, \text{m}, \text{cmt})$ and $(M_1, M_2, M_3) = (\text{cmt}, \text{m}, \text{pk})$. Both combinations fail to achieve quantum-annoyance for user exposure and signature forgeries. However, when cmt is part of the prefix we achieve $\frac{2}{3}$ -quantum-annoyance for signature forgeries, which is strictly better, since $\log_\ell \log_2 \ell < \frac{2}{3}$ for any ℓ . Note that $\frac{2}{3}$ -quantum-annoyance is achieved only in the case of messages with unbounded length. For messages of finite length, the parameter ϵ is expected to be closer to one.

Additionally, we conjecture that the four possible orders from this section to be strongly- ϵ -quantum-annoying with an ϵ close to $2/3$. If this conjecture were false, it would imply that there exists an adversary who invests significant effort in the offline phase while achieving a short online runtime (i.e., $T^{\text{onl}}[\mathcal{A}] \ll$

$\Omega(\ell^{2/3} 2^{n/3})$) while being still successful. However, this is really challenging since the adversary can not generate her own commitments and only receives them in the online phase. Therefore we conclude this section by recommending the use of Fiat-Shamir signatures where the commitment is hashed before the message.

8 Consequences for Dilithium

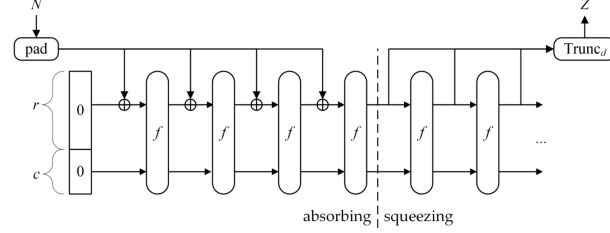


Fig. 11: This figure (taken from [52]) shows the sponge construction which is used for hash functions of the SHA-3 family where the so-called sponge function f is the Keccak function [10].

CRYSTALS-Dilithium [29] is a lattice-based signature scheme, which is one of the winners of the NIST competition for post-quantum cryptographic standards [46]. This scheme is a nested Fiat-Shamir signature scheme with challenge $\text{ch} = H(H(H(\text{pk}), m), \text{cmt})$, where the hash function $H = \text{SHAKE-256}$ [52] (a member of the SHA-3 family) relies on the sponge construction (cf. Fig. 11) and the commitment $\text{cmt} = \text{HighBits}(Ay)$ is a matrix-vector-product. In particular, A is derived from the public key which makes the commitment “user-specific” whereas y is computed (pseudo-)randomly (cf. Fig. 13).

	$M_1 = m$ (R-RSSP)	$M_1 = \text{pk}, M_2 = m$ (RPSP)	$M_i = \text{cmt}, M_j = m$ for $i < j$, $ m = B \cdot 2^{n/2}$ resp. $n/3$ (R-RPSP)
ℓ -Multi-User	$\mathcal{O}(2^{n/2} \text{ resp. } n/3)$	$\mathcal{O}(\ell \cdot 2^{n/2} \text{ resp. } n/3)$	$\tilde{\mathcal{O}}(\ell \cdot 2^{n/2})$ resp. $\mathcal{O}(\ell \cdot 2^{n/3})$
ℓ -Multi-Forgery	$\mathcal{O}(\log_2 \ell \cdot 2^{n/2} \text{ resp. } n/3)$	$\mathcal{O}(\log_2 \ell \cdot 2^{n/2} \text{ resp. } n/3)$	$\tilde{\mathcal{O}}(\ell \cdot 2^{n/2})$ resp. $\mathcal{O}(\ell^{2/3} \cdot 2^{n/3})$

Fig. 12: Attacks and their classical and quantum run time for MD-based hash functions. The classical attack we refer to in the third column originates from [41].

The analysis of Dilithium requires transferring previous results and attacks for MD-based hash functions to sponge-based (cf. Fig. 12). Benedikt et al. [5] showed for a similar attack that in case of moving from MD-based to sponge-based constructions, the capacity c takes the role of the output size n . Accordingly for our attack, we can expect a run time of $\approx \mathcal{O}(2^{c/3} \text{ resp. } c/2)$, e.g., SHAKE-256 (with $r = 1088$, $c = 512$, and $n = 1600$) can be attacked in time $\approx \mathcal{O}(2^{170.67} \text{ resp. } 256)$. To give a comprehensive analysis, we again consider both modes **plain** and **nested**.

Gen(1^λ):	Sign(sk, m):
1 : $\zeta \leftarrow_{\$} \{0, 1\}^{256}$	1 : $A \leftarrow \text{ExpandA}(\rho), \mu \leftarrow \text{H}(tr m)$
2 : $(\rho, \rho', K) \leftarrow \text{H}(\zeta)$	2 : $\kappa \leftarrow 0, (z, h) \leftarrow \perp$
3 : $A \leftarrow \text{ExpandA}(\rho)$	3 : $\rho' \leftarrow \text{H}(K rnd \mu)$ where $rnd \leftarrow_{\$} \{0, 1\}^{256}$
4 : $(s_1, s_2) \leftarrow \text{ExpandS}(\rho')$	4 : while $(z, h) = \perp$ do
5 : $(t_1, t_0) \leftarrow \text{Power2Round}_q(As_1 + s_2, d)$	5 : $y = \text{ExpandMask}(\rho', \kappa)$
6 : $tr \leftarrow \text{H}(\rho t_1)$	6 : $w = \text{HighBits}_q(Ay)$
7 : $\text{pk} \leftarrow (\rho, t_1), \text{sk} \leftarrow (\rho, K, tr, s_1, s_2, t_0)$	7 : $\tilde{c} \leftarrow \text{H}(\mu \text{Encode}(w))$
8 : return (pk, sk)	8 : $c \leftarrow \text{SampleInBall}(\tilde{c})$
	9 : $z \leftarrow y + cs_1$
	10 : $r_0 \leftarrow \text{LowBits}_q(w - cs_2)$
	11 : if $\ z\ _\infty \geq \gamma_1 - \beta$ or $\ r_0\ _\infty \geq \gamma_2 - \beta$
	12 : then $(z, h) \leftarrow \perp$
	13 : else
	14 : $h \leftarrow \text{MakeHint}_q$
	$(-ct_0, w - c(s_2 + t_0))$
	15 : if $\ ct_0\ _\infty \geq \gamma_2$ or $wt(h) \geq \omega$
	16 : then $(z, h) \leftarrow \perp$
	17 : $\kappa \leftarrow \kappa + \ell$
	18 : endwhile
	19 : return $\sigma = (\tilde{c}, z, h)$
Vrfy(pk, m, $\sigma = (\tilde{c}, z, h)$):	
1 : $A \leftarrow \text{ExpandA}(\rho)$	
2 : $\mu \leftarrow \text{H}(\text{H}(\rho t_1) m)$	
3 : $c \leftarrow \text{SampleInBall}(\tilde{c})$	
4 : $w \leftarrow \text{UseHint}(h, Az - ct_1 \cdot 2^d, 2\gamma_2)$	
5 : return $\ z\ _\infty < \gamma_1 - \beta$	
and $\tilde{c} = \text{H}(\mu w)$	
and $wt(h) \leq \omega$	

Fig. 13: ML-DSA [54], or NIST’s standardized version of CRYSTALS-Dilithium [29], where we highlighted the Fiat-Shamir related parts in gray.

8.1 Preliminaries

Let $f : \{0, 1\}^r \times \{0, 1\}^c \rightarrow \{0, 1\}^{r+c}$ be the sponge function with rate $r \in \mathbb{N}$ and capacity $c \in \mathbb{N}$. We denote by $f^{(r)}$ resp. $f^{(c)} : \{0, 1\}^{r+c} \rightarrow \{0, 1\}^r$ resp. $\{0, 1\}^c$ the “rate-part” (resp. “capacity-part”) of the sponge function, i.e., we have $f(m, y) = f^{(r)}(m, y) || f^{(c)}(m, y)$ for any $(m, y) \in \{0, 1\}^{r+c}$. Let $\text{pad} : \mathbb{N} \rightarrow \{0, 1\}^{\leq r}$ be the padding, and let f^* be the *iterated sponge function* which is defined as follows:

$$f^*(m) := y_\ell^r || y_\ell^c, \text{ where } y_i = y_i^r || y_i^c = \begin{cases} 0^{r+c} & \text{for } i = 0 \\ f(y_{i-1}^r \oplus m_i, y_{i-1}^c) & \text{for } i \in \{1, \dots, \ell\} \end{cases}$$

and $\{y_0^r, \dots, y_\ell^r\} \subset \{0, 1\}^r, \{y_0^c, \dots, y_\ell^c\} \subset \{0, 1\}^c$,

where $m = m_1 || \dots || m_\ell$, $m_i \in \{0, 1\}^r$. We call $y_{i-1}^r \oplus m_i$ the input bits and y_{i-1}^c the internal state. Then we can easily write a sponge-based hash function with one round of squeezing—which is true for Dilithium—as $\text{H}(m) := \text{Trunc}_d(f^*(m || \text{pad}(|m|)))$, $d \leq r$, where the function Trunc_d truncates the given string to the first d bits.

Moreover, we make an assumption about “well-behaving” sponge functions which is comparable to the balancedness of compression functions (cf. Def. 2). Namely, we assume the sponge function to be a β -balanced permutation:

Definition 22 (β -Balanced Sponges). *Let $f : \{0, 1\}^{r+c} \rightarrow \{0, 1\}^{r+c}$, $r \geq c$ (true for Dilithium), be the sponge function of a sponge-based hash function H . Then f is called β -balanced, where $\beta \in (0, 1]$, if and only if for any $y \in \{0, 1\}^c$ and any $z \in \{0, 1\}^r$ the number of preimages of the capacity-part $f^{(c)}$ satisfies $|(f_y^{(c)})^{-1}(z)| \geq \beta \cdot 2^{r-c}$.*

Note that this is a reasonable property since a permutation f has $|f^{-1}(z)| = 2^r$ preimages for any $z \in \{0, 1\}^{r+c}$. Thus given a fixed internal state $y \in \{0, 1\}^c$, 2^{r-c} of these preimages are expected to have capacity-part y , if f is a random permutation—and f is $\frac{1}{2}$ -balanced with overwhelming probability for $r \gg c$.

8.2 Where to Place the Message?

Let us consider combinations with $M_1 = m$. In the case of sponge-based hash functions it is not enough to only find a message pair s.th. the subsequent internal states collide. After processing the messages, only the capacity-parts are equal—not the rate-parts. Accordingly, a “usual” collision under $f^{(c)}$ needs to be modified to become a collision of the hash function, e.g., consider the following adversary:

$\mathcal{A}_{\text{R-RSSP}}^{\text{off}}(1^\lambda):$	$\mathcal{A}_{\text{R-RSSP}}^{\text{onl}}((P', P), S):$
1 : $(P_1, P'_1) \leftarrow \text{collision of } f_{0^c}^{(c)}(\cdot)$ 2 : $P_2 \leftarrow f^{(r)}(P_1, 0^c)$ and $P'_2 \leftarrow f^{(r)}(P'_1, 0^c)$ 3 : return $(P' \leftarrow P'_1 P'_2, P \leftarrow P_1 P_2)$	1 : return P'

For the run time, we can classically search for a collision in $\mathcal{O}(2^{c/2})$ steps via a birthday attack, and in the quantum case in $\mathcal{O}(2^{c/3})$ steps via the $\widetilde{\text{BHT}}$ -algorithm below, which is the BHT-algorithm adapted to sponge functions (cf. Prop. 23). In both settings, the online phase requires constant time $\mathcal{O}(1)$.

Proposition 23 ((Adapted) $\widetilde{\text{BHT}}$ -Algorithm [13]). *Let $y \in \{0, 1\}^c$ be an internal state, and $f^{(c)}$ be the capacity-part of a sponge function f . Then there exists a quantum algorithm making $\mathcal{O}(2^{c/3})$ quantum queries to $f^{(c)}$ to output a collision of the function $f_y^{(c)}(\cdot)$, i.e., two message blocks $(m, m') \in \{0, 1\}^r \times \{0, 1\}^r$ such that $f_y^{(c)}(m) := f^{(c)}(m, y) = f^{(c)}(m', y) = f_y^{(c)}(m')$.*

Proof. Proof works as for Prop. 12: Given $y \in \{0, 1\}^c$, sample $\{m_i\}_{i=1}^k \subset \{0, 1\}^r$ and run $\text{Grover}(F, 1)$ (cf. Thm. 13), where $F : \{0, 1\}^r \rightarrow \{0, 1\}$ is defined as

$$F(m) = 1 \iff \exists i \in \{1, \dots, k\} \text{ such that } f_y^{(c)}(m) = f_y^{(c)}(m_i) \text{ and } m \neq m_i.$$

The presampling runs in $\mathcal{O}(k)$, Grover in $\mathcal{O}(\sqrt{N/t})$ where $N = 2^r$ and $t = |F^{-1}(1)| \geq (\beta \cdot 2^{r-c} - 1) \cdot k$ since f is β -balanced. The overall run time is $\mathcal{O}(k + (\beta \cdot k)^{-1/2} \cdot 2^{c/2})$, and $\mathcal{O}(2^{c/3})$ for $k = \beta^{-1/3} \cdot 2^{c/3}$ and constant β . $\square$

The adversary from above is successful as a simple computation shows:

$$\begin{aligned} H(P) &= f^*(P_1 || P_2) = f(f^{(r)}(P_1, 0^c) \oplus P_2, f_{0^c}^{(c)}(P_1)) = f(0^r, f_{0^c}^{(c)}(P_1')) \\ &= f(f^{(r)}(P_1', 0^c) \oplus P_2', f_{0^c}^{(c)}(P_1')) = H(P'). \end{aligned}$$

Similar to Sec. 5, we run an ℓ -multi-user attack in time $\mathcal{O}(2^{c/2} \text{ resp. } c/3)$ by running $\mathcal{A}_{\text{R-RSSP}}^{\text{off}}$ once and an ℓ -multi-forgery attack in time $\mathcal{O}(\log_2 \ell \cdot 2^{c/2} \text{ resp. } c/3)$ by constructing Joux's multi-collisions.



Fig. 14: Joux's multi-collision [39] of $f^{(c)}$, i.e., we have an edge $p \in \{0, 1\}^r$ between nodes $x, y \in \{0, 1\}^c$ if $f^{(c)}(p, x) = y$.

Similar to before, it is sufficient to find multi-collisions of the capacity-part $f^{(c)}$ (cf. Fig. 14). However, these multi-collisions must be processed to obtain a forgery. Consider the following algorithms:

$\mathcal{A}_\ell(1^\lambda, \{\text{pk}_i, \text{Sign}_{\text{sk}_i}(\cdot)\}_{i=1}^u, \text{mode})$:	Process(P):
1 : Build a $\lceil \log_2 \ell \rceil + 1$ (Joux's) multi-collision	1 : Parse P as $p_1 \dots p_L$
2 : $\sigma \leftarrow \text{Sign}_{\text{sk}_1}(P)$,	2 : s.th. $ p_i = r$
3 : where $P \leftarrow \text{Process}(p_1^{(0)} \dots p_{\lceil \log_2 \ell \rceil + 1}^{(0)})$	3 : $P_1 \leftarrow p_1$ and $y_1 \leftarrow 0^c$
4 : for $i \in \{1, \dots, \ell\}$ do	4 : for $i = \{2, \dots, \ell\}$ do
5 : $i_1 i_2 \dots i_{\lceil \log_2 \ell \rceil + 1} \leftarrow \text{binary encoding of } i$	5 : $P_i \leftarrow f^{(r)}(p_{i-1}, y_{i-1}) \oplus p_i$
6 : $P_i \leftarrow \text{Process}(p_1^{(i_1)} \dots p_{\lceil \log_2 \ell \rceil + 1}^{(i_{\lceil \log_2 \ell \rceil + 1})})$	6 : $y_i \leftarrow f^{(c)}(p_{i-1}, y_{i-1})$
7 : return $\{(P_i, \sigma, 1)\}_{i=1}^\ell$	7 : $P_{\ell+1} \leftarrow f^{(r)}(p_\ell, y_\ell)$
	8 : return $P_1 \dots P_{\ell+1}$

Note, that the algorithm **Process** is a generalization of the prefix processing that happens in the R-RSSP attack of this section. It follows:

Corollary 24. *Any [plain/nested] Fiat-Shamir signature scheme—using a sponge-based hash function and starting with $M_1 = m$ —is neither [classical/quantum]-annoying for user exposures nor signature forgeries.*

Therefore:

1st RULE: No matter if H is MD- or sponge-based, do not start with m .

8.3 How to Order Message and Commitment?

Next, we consider $(M_1, M_2, M_3) = (\text{pk}, m, \text{cmt})$. In that case, we proceed as in Sec. 6. Andreeva et al.'s [3] construction helps to run an ℓ -multi-user attack

in time $\mathcal{O}(\ell \cdot 2^{c/2} \text{ resp. } c/3)$, and Joux’s multi-collisions to run an ℓ -multi-forgery attack in time $\mathcal{O}(\log_2 \ell \cdot 2^{c/2} \text{ resp. } c/3)$. It follows immediately:

Corollary 25. *Any [plain/nested] Fiat-Shamir signature scheme of the form $(M_1, M_2, M_3) = (\text{pk}, m, \text{cmt})$ —using a sponge-based hash function—is not [classical/ quantum]-annoying for signature forgeries.*

Note that in case the ℓ -multi-user adversary with run time $\mathcal{O}(\ell \cdot 2^{c/2} \text{ resp. } c/3)$ does *not* imply [classical|quantum]-annoyance for user exposures. The reason is, that—similar to before, where we needed lower bounds for random functions to prove annoyance for MD-based constructions—we require lower bounds for random permutations for the sponge-based constructions which—to our knowledge—are unknown. However—in our opinion—this is not so relevant of the analysis for $(M_1, M_2, M_3) = (\text{pk}, m, \text{cmt})$ since Cor. 25 is reason enough to conclude:

2nd RULE: No matter if H is MD- or sponge-based, put cmt before m.

8.4 How to Order in Dilithium

In this section, we will argue why we should use one of the following combinations:

- (1) $M_1 = \text{pk}, M_2 = \text{cmt}, M_3 = m,$ (3) $M_1 = \text{cmt}, M_2 = m, M_3 = \text{pk},$ and
- (2) $M_1 = \text{cmt}, M_2 = \text{pk}, M_3 = m,$ (4) $M_1 = \text{cmt}, M_2 = m.$

Remember, that we argued in Sec. 7 that these combinations correspond to a *salted* version of the hash function. Since the adversary, in general, can not come up with own commitments, the commitment must be given by the challenger and appears as a random prefix to the adversary which determines the *last input hash state* before the sponge processes the message. This makes the combination above comparable to a sponge construction with *randomized IV*. Accordingly forging a signature (of m) which has used a randomized IV ($f^*(P)$) means finding a second preimage $m' \neq m$.

...in the Classical Setting. Unfortunately, we are unaware of classical tight bounds of finding (multiple) second preimage(s). However for 1- and 2-block messages a lower bound for finding a single second preimage is known. Remember that every found second preimage is also a collision of the sponge function:

Theorem 26 ([1]). *Let $(\mathcal{A}_1, \mathcal{A}_2)$ a two-staged adversary parametrized by (space) S and (time) T and H a sponge-based hash function. $\mathcal{A}_1$ computes an S -bit advice for $\mathcal{A}_2$. $\mathcal{A}_2$ gets the advice and a random IV, makes at most T queries to h , and attempts to find a B -block collision (m, m') for H . Both $\mathcal{A}_1$ and $\mathcal{A}_2$ are unbounded in the running time. The advantage is bounded by*

$$- \tilde{\mathcal{O}}(S^2 T^2 / 2^{2c} + S / 2^c + T / 2^c + T^2 / 2^r) \text{ for } B = 1 \text{ and}$$

$$- \tilde{\mathcal{O}}(ST/2^c + S^2T^4/2^{2c} + T^2/\min(2^c, 2^r)) \text{ for } B = 2.$$

Assuming that $\mathcal{A}_1$ runs in time $\Omega(S)$, we get the best overall running time if $S \approx T - \mathcal{O}(\min(2^{c/2}, 2^{r/2}))$ for $B = 1$, $\mathcal{O}(2^{c/3})$ for $B = 2$, and $\mathcal{O}(2^{c/2}$ resp. $c/3)$ for Dilithium. Note that it is unclear how this extends to the case of $B \geq 3$ or finding multiple second preimages.

However since every generation of a signature corresponds to an usage of a new randomized IV in the sponge design, we expect that the combination of this section are indeed (strongly-) ε -annoyant with ε close to one.

In case of unbounded collision message sizes, Kelsey and Schneier’s [41] second preimage attack is the best known attack which can be used for multi-user and multi-forgery attacks in the classical setting with run time $\tilde{\mathcal{O}}(\ell \cdot 2^{c/2})$.

...in the Quantum Setting. Adapting our R-RPSP attack from Sec 7.2, we achieve $\mathcal{O}((\ell \text{ resp. } \ell^{2/3}) \cdot 2^{c/3})$ for quantum multi-user resp. multi-forgery attacks. This result leads to the conclusion that the schemes (1)-(4) are *not* quantum-annoying for signature forgeries. However we can not make any definitive statement about the quantum-annoyance for user exposures, as no known lower quantum bounds about finding (multiple) second preimage(s). Only in the context of 1-round sponges, there is a bound of $\tilde{\Omega}(2^{c/3})$ by [47] known which does not fit the use case of Dilithium.

Nevertheless, we believe the combinations (1)-(4) are preferable since the best-known multi-forgery attack on $(M_1 = \text{pk}, M_2 = \text{m}, M_3 = \text{cmt})$ takes less time than the one on combinations of this section, which can be easily mitigated by suggesting a maximum message length.

It is important to note that our analysis is based on the attacks we have identified. While we have not been able—in contrast to MD-based hash functions—to prove ε -[classical|quantum]-annoyance for any ε , this does not preclude the existence of better attacks. However, any attack from this section is likely to also work in the previous section—though not necessarily the other way around. We therefore recommend using one of the stated combinations above.

Finally, we want to mention that ML-DSA is using the combination $(M_1 = \text{pk}, M_2 = \text{m}, M_3 = \text{cmt})$. We assume that this order is used for the sake of saving computational effort during the rejection sampling (cf. line 12 and 16 in Fig. 13): The algorithm samples new candidates for cmt , until certain parameters are *in a certain range* (cf. line 11 and 15). As the challenge is of the form $H(H(H(\text{pk})||\text{m})||\text{cmt})$, the algorithm only needs to redo the outermost hash evaluation for every freshly sampled cmt . Accordingly, as the inner hash evaluations of pk and m remain the same, the algorithm saves computations if it process the commitment cmt as last. However,—considering *other nested* Fiat-Shamir signature schemes than defined in Def. 1, e.g., $H(M_1||H(H(M_2)||M_3))$ or $H(M_1||H(M_2||H(M_3)))$ —schemes with hashing order (1)-(4) become *equally good* to $H(H(H(\text{pk})||\text{m})||\text{cmt})$ (ML-DSA) in terms of computational effort. To our knowledge, there is no particular reason for the choice “of nesting”, and it may be worth reconsidering the form of challenge and the hashing order to prevent a possible RPSP attack.

Acknowledgments

We thank the anonymous reviewers for valuable comments.

This research work has been funded by the German Federal Ministry of Education and Research and the Hessian Ministry of Higher Education, Research, Science and the Arts within their joint support of the National Research Center for Applied Cybersecurity ATHENE.

References

1. Akshima, Duan, X., Guo, S., Liu, Q.: On time-space lower bounds for finding short collisions in sponge hash functions. In: Rothblum, G.N., Wee, H. (eds.) TCC 2023, Part III. LNCS, vol. 14371, pp. 237–270. Springer, Cham (Nov / Dec 2023). https://doi.org/10.1007/978-3-031-48621-0_9
2. Alkim, E., Bindel, N., Buchmann, J., Dagdelen, Ö., Eaton, E., Gutoski, G., Krämer, J., Pawlega, F.: Revisiting tesla in the quantum random oracle model. In: Post-Quantum Cryptography: 8th International Workshop, PQCrypto 2017, Utrecht, The Netherlands, June 26–28, 2017, Proceedings 8. pp. 143–162. Springer (2017)
3. Andreeva, E., Bouillaguet, C., Dunkelman, O., Kelsey, J.: Herding, second preimage and trojan message attacks beyond Merkle-Damgård. In: Jacobson, Jr., M.J., Rijmen, V., Safavi-Naini, R. (eds.) SAC 2009. LNCS, vol. 5867, pp. 393–414. Springer, Berlin, Heidelberg (Aug 2009). https://doi.org/10.1007/978-3-642-05445-7_25
4. Bellare, M., Davis, H., Günther, F.: Separate your domains: NIST PQC KEMs, oracle cloning and read-only indistinguishability. In: Canteaut, A., Ishai, Y. (eds.) EUROCRYPT 2020, Part II. LNCS, vol. 12106, pp. 3–32. Springer, Cham (May 2020). https://doi.org/10.1007/978-3-030-45724-2_1
5. Benedikt, B.J., Fischlin, M., Huppert, M.: Nostradamus goes quantum. In: Agrawal, S., Lin, D. (eds.) ASIACRYPT 2022, Part III. LNCS, vol. 13793, pp. 583–613. Springer, Cham (Dec 2022). https://doi.org/10.1007/978-3-031-22969-5_20
6. Bernhard, D., Pereira, O., Warinschi, B.: How not to prove yourself: Pitfalls of the Fiat-Shamir heuristic and applications to Helios. In: Wang, X., Sako, K. (eds.) ASIACRYPT 2012. LNCS, vol. 7658, pp. 626–643. Springer, Berlin, Heidelberg (Dec 2012). https://doi.org/10.1007/978-3-642-34961-4_38
7. Bernstein, D.J.: Multi-user Schnorr security, revisited. Cryptology ePrint Archive, Report 2015/996 (2015), <https://eprint.iacr.org/2015/996>
8. Bernstein, D.J., Duif, N., Lange, T., Schwabe, P., Yang, B.Y.: High-speed high-security signatures. In: Preneel, B., Takagi, T. (eds.) CHES 2011. LNCS, vol. 6917, pp. 124–142. Springer, Berlin, Heidelberg (Sep / Oct 2011). https://doi.org/10.1007/978-3-642-23951-9_9
9. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: Sponge functions. Ecrypt Hash Workshop (2007)
10. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: Keccak. In: Johansson, T., Nguyen, P.Q. (eds.) EUROCRYPT 2013. LNCS, vol. 7881, pp. 313–314. Springer, Berlin, Heidelberg (May 2013). https://doi.org/10.1007/978-3-642-38348-9_19

11. Boneh, D., Dagdelen, Ö., Fischlin, M., Lehmann, A., Schaffner, C., Zhandry, M.: Random oracles in a quantum world. In: Lee, D.H., Wang, X. (eds.) ASIACRYPT 2011. LNCS, vol. 7073, pp. 41–69. Springer, Berlin, Heidelberg (Dec 2011). https://doi.org/10.1007/978-3-642-25385-0_3
12. Boyer, M., Brassard, G., Høyer, P., Tapp, A.: Tight bounds on quantum searching. *Fortschritte der Physik* **46**(4-5), 493–505 (1998)
13. Brassard, G., Høyer, P., Tapp, A.: Quantum cryptanalysis of hash and claw-free functions. In: Lucchesi, C.L., Moura, A.V. (eds.) LATIN '98: Theoretical Informatics, Third Latin American Symposium, Campinas, Brazil, April, 20-24, 1998, Proceedings. Lecture Notes in Computer Science, vol. 1380, pp. 163–169. Springer (1998)
14. Brown, D.R.L.: A flaw in a theorem about Schnorr signatures. Cryptology ePrint Archive, Report 2015/509 (2015), <https://eprint.iacr.org/2015/509>
15. Brown, D.R.L.: Short Schnorr signatures require a hash function with more than just random-prefix resistance. Cryptology ePrint Archive, Report 2015/169 (2015), <https://eprint.iacr.org/2015/169>
16. Carolan, J., Poremba, A.: Quantum one-wayness of the single-round sponge with invertible permutations. In: Reyzin, L., Stebila, D. (eds.) CRYPTO 2024, Part VI. LNCS, vol. 14925, pp. 218–252. Springer, Cham (Aug 2024). https://doi.org/10.1007/978-3-031-68391-6_7
17. CEN: Application interface for smart cards used as secure signature creation devices - part 1: Basic requirements. Norm DIN EN 419212-1:2015-03, EUROPEAN COMMITTEE FOR STANDARDIZATION (March 2015)
18. Chase, M., Derler, D., Goldfeder, S., Orlandi, C., Ramacher, S., Rechberger, C., Slamanig, D., Zaverucha, G.: Post-quantum zero-knowledge and signatures from symmetric-key primitives. In: Thuraisingham, B.M., Evans, D., Malkin, T., Xu, D. (eds.) ACM CCS 2017. pp. 1825–1842. ACM Press (Oct / Nov 2017). <https://doi.org/10.1145/3133956.3133997>
19. Chen, M.S., Hülsing, A., Rijneveld, J., Samardjiska, S., Schwabe, P.: From 5-pass MQ-based identification to MQ-based signatures. In: Cheon, J.H., Takagi, T. (eds.) ASIACRYPT 2016, Part II. LNCS, vol. 10032, pp. 135–165. Springer, Berlin, Heidelberg (Dec 2016). https://doi.org/10.1007/978-3-662-53890-6_5
20. Coron, J.S., Dodis, Y., Malinaud, C., Puniya, P.: Merkle-Damgård revisited: How to construct a hash function. In: Shoup, V. (ed.) CRYPTO 2005. LNCS, vol. 3621, pp. 430–448. Springer, Berlin, Heidelberg (Aug 2005). https://doi.org/10.1007/11535218_26
21. Cremers, C., Düzlül, S., Fiedler, R., Fischlin, M., Janson, C.: BUFFing signature schemes beyond unforgeability and the case of post-quantum signatures. In: 2021 IEEE Symposium on Security and Privacy. pp. 1696–1714. IEEE Computer Society Press (May 2021). <https://doi.org/10.1109/SP40001.2021.00093>
22. Czapkowski, J., Bruinderink, L.G., Hülsing, A., Schaffner, C., Unruh, D.: Post-quantum security of the sponge construction. In: Lange, T., Steinwandt, R. (eds.) Post-Quantum Cryptography - 9th International Conference, PQCrypto 2018. pp. 185–204. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-79063-3_9
23. Czapkowski, J., Hülsing, A., Schaffner, C.: Quantum indistinguishability of random sponges. In: Boldyreva, A., Micciancio, D. (eds.) CRYPTO 2019, Part II. LNCS, vol. 11693, pp. 296–325. Springer, Cham (Aug 2019). https://doi.org/10.1007/978-3-030-26951-7_11

24. Czapkowski, J., Majenz, C., Schaffner, C., Zur, S.: Quantum lazy sampling and game-playing proofs for quantum indistinguishability. Cryptology ePrint Archive, Report 2019/428 (2019), <https://eprint.iacr.org/2019/428>
25. Damgård, I.: A design principle for hash functions. In: Brassard, G. (ed.) CRYPTO'89. LNCS, vol. 435, pp. 416–427. Springer, New York (Aug 1990). https://doi.org/10.1007/0-387-34805-0_39
26. Dinur, I.: Tight time-space lower bounds for finding multiple collision pairs and their applications. In: Canteaut, A., Ishai, Y. (eds.) Advances in Cryptology - EUROCRYPT 2020 - 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part I. Lecture Notes in Computer Science, vol. 12105, pp. 405–434. Springer (2020). https://doi.org/10.1007/978-3-030-45721-1_15, https://doi.org/10.1007/978-3-030-45721-1_15
27. Dodis, Y., Guo, S., Katz, J.: Fixing cracks in the concrete: Random oracles with auxiliary input, revisited. In: Coron, J.S., Nielsen, J.B. (eds.) EUROCRYPT 2017, Part II. LNCS, vol. 10211, pp. 473–495. Springer, Cham (Apr / May 2017). https://doi.org/10.1007/978-3-319-56614-6_16
28. Dodis, Y., Ristenpart, T., Shrimpton, T.: Salvaging Merkle-Damgård for practical applications. In: Joux, A. (ed.) EUROCRYPT 2009. LNCS, vol. 5479, pp. 371–388. Springer, Berlin, Heidelberg (Apr 2009). https://doi.org/10.1007/978-3-642-01001-9_22
29. Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS-Dilithium: A lattice-based digital signature scheme. IACR TCHES **2018**(1), 238–268 (2018). <https://doi.org/10.13154/tches.v2018.i1.238-268>, <https://tches.iacr.org/index.php/TCHES/article/view/839>
30. Eaton, E., Stebila, D.: The “quantum annoying” property of password-authenticated key exchange protocols. In: Cheon, J.H., Tillich, J.P. (eds.) Post-Quantum Cryptography - 12th International Workshop, PQCrypto 2021. pp. 154–173. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-81293-5_9
31. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Odlyzko, A.M. (ed.) CRYPTO'86. LNCS, vol. 263, pp. 186–194. Springer, Berlin, Heidelberg (Aug 1987). https://doi.org/10.1007/3-540-47721-7_12
32. FIPS, P.: 180-4 (2011)
33. Galbraith, S.D., Malone-Lee, J., Smart, N.P.: Public key signatures in the multi-user setting. Inf. Process. Lett. **83**(5), 263–266 (2002). [https://doi.org/10.1016/S0020-0190\(01\)00338-6](https://doi.org/10.1016/S0020-0190(01)00338-6), [https://doi.org/10.1016/S0020-0190\(01\)00338-6](https://doi.org/10.1016/S0020-0190(01)00338-6)
34. Ghoshal, A., Komargodski, I.: On time-space tradeoffs for bounded-length collisions in merkle-Damgård hashing. In: Dodis, Y., Shrimpton, T. (eds.) CRYPTO 2022, Part III. LNCS, vol. 13509, pp. 161–191. Springer, Cham (Aug 2022). https://doi.org/10.1007/978-3-031-15982-4_6
35. Grover, L.K.: A fast quantum mechanical algorithm for database search. In: Miller, G.L. (ed.) Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing, Philadelphia, Pennsylvania, USA, May 22–24, 1996. pp. 212–219. ACM (1996)
36. Hamoudi, Y., Magniez, F.: Quantum time-space tradeoff for finding multiple collision pairs. ACM Trans. Comput. Theory **15**(1-2), 1–22 (2023). <https://doi.org/10.1145/3589986>, <https://doi.org/10.1145/3589986>
37. ISO: Information technology. security techniques. digital signatures with appendix. discrete logarithm based mechanisms. Norm ISO/IEC 14888-3:2016-03 (E), International Organization for Standardization (March 2016)

38. Josefsson, S., Liusvaara, I.: Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032 (Jan 2017). <https://doi.org/10.17487/RFC8032>, <https://www.rfc-editor.org/info/rfc8032>
39. Joux, A.: Multicollisions in iterated hash functions. Application to cascaded constructions. In: Franklin, M. (ed.) CRYPTO 2004. LNCS, vol. 3152, pp. 306–316. Springer, Berlin, Heidelberg (Aug 2004). https://doi.org/10.1007/978-3-540-28628-8_19
40. Kelsey, J., Kohno, T.: Herding hash functions and the Nostradamus attack. In: Vaudenay, S. (ed.) EUROCRYPT 2006. LNCS, vol. 4004, pp. 183–200. Springer, Berlin, Heidelberg (May / Jun 2006). https://doi.org/10.1007/11761679_12
41. Kelsey, J., Schneier, B.: Second preimages on n -bit hash functions for much less than 2^n work. In: Cramer, R. (ed.) EUROCRYPT 2005. LNCS, vol. 3494, pp. 474–490. Springer, Berlin, Heidelberg (May 2005). https://doi.org/10.1007/11426639_28
42. Kiltz, E., Masny, D., Pan, J.: Optimal security proofs for signatures from identification schemes. In: Robshaw, M., Katz, J. (eds.) CRYPTO 2016, Part II. LNCS, vol. 9815, pp. 33–61. Springer, Berlin, Heidelberg (Aug 2016). https://doi.org/10.1007/978-3-662-53008-5_2
43. Kortelainen, T., Kortelainen, J.: On diamond structures and trojan message attacks. In: Sako, K., Sarkar, P. (eds.) ASIACRYPT 2013, Part II. LNCS, vol. 8270, pp. 524–539. Springer, Berlin, Heidelberg (Dec 2013). https://doi.org/10.1007/978-3-642-42045-0_27
44. Liu, Q., Zhandry, M.: On finding quantum multi-collisions. In: Ishai, Y., Rijmen, V. (eds.) EUROCRYPT 2019, Part III. LNCS, vol. 11478, pp. 189–218. Springer, Cham (May 2019). https://doi.org/10.1007/978-3-030-17659-4_7
45. Lyubashevsky, V.: Fiat-Shamir with aborts: Applications to lattice and factoring-based signatures. In: Matsui, M. (ed.) ASIACRYPT 2009. LNCS, vol. 5912, pp. 598–616. Springer, Berlin, Heidelberg (Dec 2009). https://doi.org/10.1007/978-3-642-10366-7_35
46. Lyubashevsky, V., Ducas, L., Kiltz, E., Lepoint, T., Schwabe, P., Seiler, G., Stehlé, D., Bai, S.: CRYSTALS-DILITHIUM. Tech. rep., National Institute of Standards and Technology (2022), available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>
47. Majenz, C., Malavolta, G., Walter, M.: Permutation superposition oracles for quantum query lower bounds. Cryptology ePrint Archive, Report 2024/1140 (2024), <https://eprint.iacr.org/2024/1140>
48. Maurer, U.M., Renner, R., Holenstein, C.: Indifferentiability, impossibility results on reductions, and applications to the random oracle methodology. In: Naor, M. (ed.) TCC 2004. LNCS, vol. 2951, pp. 21–39. Springer, Berlin, Heidelberg (Feb 2004). https://doi.org/10.1007/978-3-540-24638-1_2
49. Menezes, A., Smart, N.P.: Security of signature schemes in a multi-user setting. Des. Codes Cryptogr. **33**(3), 261–274 (2004). <https://doi.org/10.1023/B:DESI.0000036250.18062.3F>, <https://doi.org/10.1023/B:DESI.0000036250.18062.3f>
50. Merkle, R.C.: A certified digital signature. In: Brassard, G. (ed.) CRYPTO’89. LNCS, vol. 435, pp. 218–238. Springer, New York (Aug 1990). https://doi.org/10.1007/0-387-34805-0_21
51. Neven, G., Smart, N.P., Warinschi, B.: Hash function requirements for schnorr signatures. Journal of Mathematical Cryptology **3**(1), 69–87 (2009). <https://doi.org/doi:10.1515/JMC.2009.004>, <https://doi.org/10.1515/JMC.2009.004>

52. NIST: NIST SHA-3 Competition, <http://csrc.nist.gov/groups/ST/hash/sha-3/index.html>
53. NIST: Sha-3 standard: Permutation-based hash and extendable-output functions. Norm FIPS 202, National Institute of Standards and Technology (August 2015)
54. NIST: Module-lattice-based digital signature standard. Norm FIPS 204, National Institute of Standards and Technology (August 2024)
55. van Oorschot, P.C., Wiener, M.J.: Parallel collision search with cryptanalytic applications. *J. Cryptol.* **12**(1), 1–28 (1999). <https://doi.org/10.1007/PL00003816>, <https://doi.org/10.1007/PL00003816>
56. Pornin, T., Stern, J.P.: Digital signatures do not guarantee exclusive ownership. In: Ioannidis, J., Keromytis, A., Yung, M. (eds.) *ACNS 2005*. LNCS, vol. 3531, pp. 138–150. Springer, Berlin, Heidelberg (Jun 2005). https://doi.org/10.1007/11496137_10
57. Schnorr, C.P.: Efficient identification and signatures for smart cards (abstract) (rump session). In: Quisquater, J.J., Vandewalle, J. (eds.) *EUROCRYPT’89*. LNCS, vol. 434, pp. 688–689. Springer, Berlin, Heidelberg (Apr 1990). https://doi.org/10.1007/3-540-46885-4_68
58. Schnorr, C.P.: Efficient signature generation by smart cards. *Journal of Cryptology* **4**(3), 161–174 (Jan 1991). <https://doi.org/10.1007/BF00196725>
59. Zhandry, M.: A note on the quantum collision and set equality problems. *Quantum Inf. Comput.* **15**(7&8), 557–567 (2015). <https://doi.org/10.26421/QIC15.7-8-2>, <https://doi.org/10.26421/QIC15.7-8-2>
60. Zhandry, M.: Redeeming reset indifferentiability and applications to post-quantum security. In: Tibouchi, M., Wang, H. (eds.) *ASIACRYPT 2021, Part I*. LNCS, vol. 13090, pp. 518–548. Springer, Cham (Dec 2021). https://doi.org/10.1007/978-3-030-92062-3_18